

Wydział Elektroniki i Technik Informacyjnych
Politechnika Warszawska

Projektowanie układów sterowania
(projekt grupowy)

Sprawozdanie z projektu i ćwiczenia laboratoryjnego
nr 3, zadanie nr 11

Piotr Patek, Kamil Cisek, Mateusz Smardzewski, Piotr
Sienkiewicz

Warszawa, 2024

Spis treści

1. Projekt	2
1.1. Sprawdzenie poprawności punktu pracy	2
1.2. Charakterystyka statyczna procesu	2
1.3. Algorytm PID i DMC	5
1.3.1. PID	5
1.3.2. DMC	6
1.4. Testy algorytmów PID i DMC na nieliniowym procesie	8
1.4.1. PID	8
1.4.2. DMC	11
1.5. Rozmyty PID i rozmyty DMC	12
1.5.1. Funkcja przynależności	12
1.5.2. Rozmyty PID	14
1.5.3. Rozmyty DMC	15
1.6. Testy rozmytych algorytmów PID i DMC	18
1.6.1. Rozmyty PID	18
1.6.2. Rozmyty DMC	21
2. Laboratorium	28
2.1. Identyfikacja i modelowanie obiektu	28
2.1.1. Ustalenie punktu pracy	28
2.1.2. Charakterystyka statyczna	29
2.1.3. Model rozmyty	31
2.2. Porównanie regulatorów rozmytych z konwencjonalnymi	32
2.2.1. Konwencjonalne regulatory PID i DMC	32
2.2.2. Implementacja regulatorów rozmytych zgodnie z podejściem PDC	34
2.2.3. Testowanie regulatorów rozmytych	42

1. Projekt

1.1. Sprawdzenie poprawności punktu pracy

Aby sprawdzić poprawność punktu pracy $u = y = 0$ dla obiektu symulacja_obiektu1y_p3 wywołano w Matlabie polecenie przedstawione na listingu 1.1.

Listing 1.1: Sprawdzenie poprawności punktu pracy

```
>> symulacja_obiektu1y_p3(0, 0, 0, 0)

ans =

    0
```

Jak widać zwracany wynik to 0, punkt pracy jest więc poprawny.

1.2. Charakterystyka statyczna procesu

Do wyznaczenia kolejnych odpowiedzi skokowych, narysowania ich na wykresie a także wyznaczenia na ich podstawie charakterystyki statycznej i narysowania jej na wykresie napisano w Matlabie program przedstawiony na listingu 1.2.

Listing 1.2: Wizualizacji odpowiedzi skokowych i charakterystyki statycznej

```
clear;

Upp = 0.0;
Ypp = 0.0;
du_max = 1.0;
n = 50;
n_plots = 5;
y_stat = zeros(1, n_plots);
u = zeros(1, n_plots);

for i=1:n_plots
    du = -du_max + 2 * du_max * (i-1) / (n_plots-1);
    y = stepResponse(Upp, Ypp, du, n);

    hold on;
    title("Odpowiedzi skokowe procesu")
    stairs(y, 'DisplayName', "Skok na u=" + (Upp + du));
    xlabel("k");
    ylabel("y(k)");

    y_stat(i) = y(n);
    u(i) = Upp + du;
end
lgd = legend;
lgd.Position = [0.65, 0.3, 0.25, 0.15];
hold off;

% Charakterystyka statyczna
figure;
```

```
title("Charakterystyka statyczna");  
plot(u, y_stat);  
xlabel("u");  
ylabel("y(u)");
```

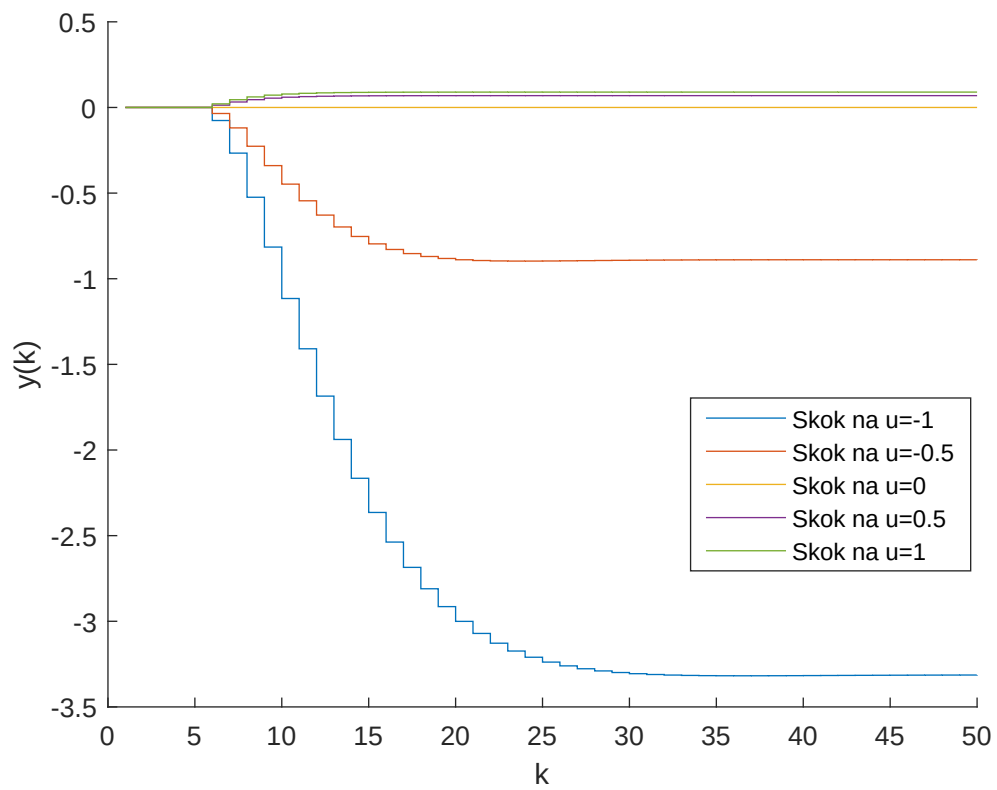
Najważniejszą częścią tego programu jest funkcja **stepResponse**, która została przedstawiona na listingu 1.3.

Listing 1.3: Funkcja stepResponse

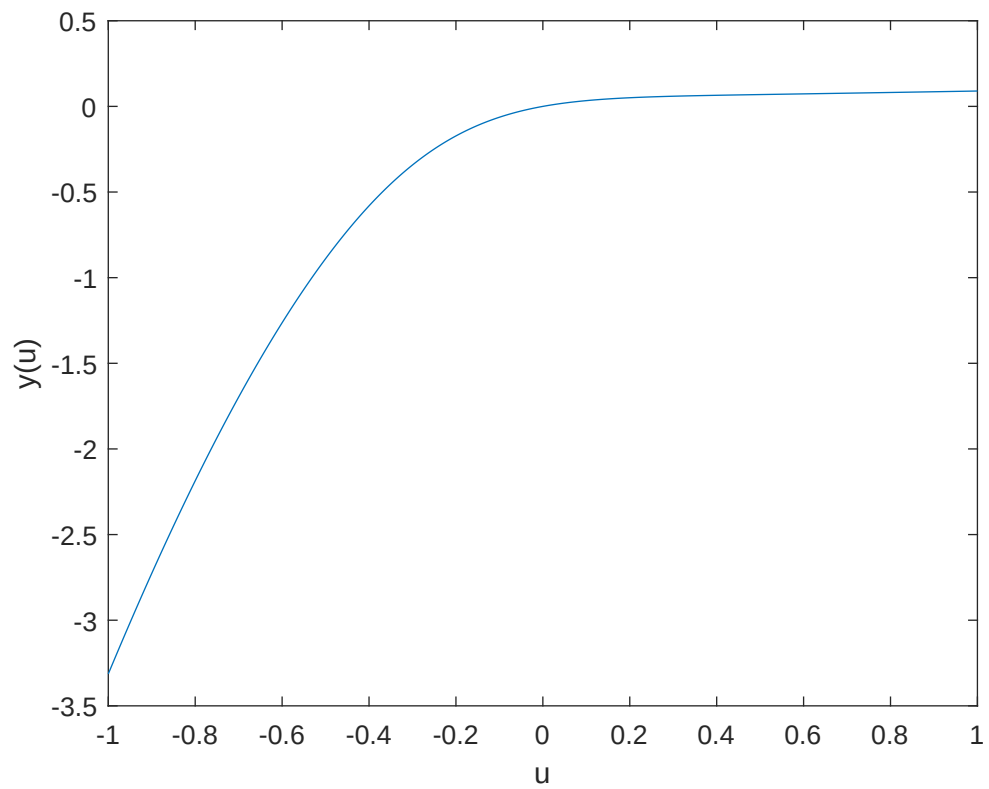
```
function y = stepResponse(Upp, Ypp, du, n)  
    u = (Upp + du) * ones(1, n);  
    y = zeros(1, n);  
  
    for i=1:n  
        if i >= 2  
            Ykm1 = y(i-1);  
        else  
            Ykm1 = Ypp;  
        end  
  
        if i >= 3  
            Ykm2 = y(i-2);  
        else  
            Ykm2 = Ypp;  
        end  
  
        if i >= 6  
            Ukm5 = u(i-5);  
        else  
            Ukm5 = Upp;  
        end  
  
        if i >= 7  
            Ukm6 = u(i-6);  
        else  
            Ukm6 = Upp;  
        end  
  
        y(i) = symulacja_obiektu11y_p3(Ukm5, Ukm6, Ykm1, Ykm2);  
    end  
end
```

Zebrane odpowiedzi skokowe zebrane na jednym rysunku w przedstawiono na rysunku 1.1. Ograniczenia sterowania obiektu to $-1 \leq u \leq 1$, więc w takim zakresie sterowania są skoki.

Widać, że wzmocnienie obiektu dla ujemnych skoków jest dużo większe co do wartości niż dla dodatnich skoków sterowania. Kształt odpowiedzi skokowych również optycznie wydaje się być lekko różny dla różnych wartości skoków, co pokazuje, że właściwości dynamiczne obiektu są nieliniowe. Wykres charakterystyki statycznej (rysunek 1.2) pokazuje, że właściwości statyczne obiektu są również silnie nieliniowe.



Rys. 1.1: Odpowiedzi skokowe procesu



Rys. 1.2: Charakterystyka statyczna procesu

1.3. Algorytm PID i DMC

W tej sekcji znajdują się kody klasycznych algorytmów PID i DMC (nie rozmytych).

1.3.1. PID

Kod algorytmu PID przedstawiono na listingu 1.4. Został on stworzony na podstawie programu z laboratorium 1.

Listing 1.4: Algorytm PID

```
clear;
% Process constants
u_min = -1;
u_max = 1;
Upp = 0;
Ypp = 0;

% Digital PID parameters
Kp = 1.925;
Ti = 3;
Td = 1;
[r2, r1, r0] = discrete_pid_parameters(Kp, Ti, Td, 0.5);

% General settings
iterations = 800;
kstart = 10;
u = ones(1, iterations) * Upp;
y = ones(1, iterations) * Ypp;
e = zeros(1, iterations);

% Set trajectory
step1_time = 10;
step2_time = 200;
step3_time = 400;
step4_time = 600;

step1_value = 0.02;
step2_value = 0.06;
step3_value = -0.05;
step4_value = -0.2;

yzad(1:step1_time) = Ypp;
yzad(step1_time:step2_time) = step1_value;
yzad(step2_time:step3_time) = step2_value;
yzad(step3_time:step4_time) = step3_value;
yzad(step4_time:iterations) = step4_value;

% Validation
e_sum = 0;

% Main loop
for k=kstart:iterations

    % Generate process output
    y(k) = symulacja_obiektu11y_p3(u(k-5), u(k-6), y(k-1), y(k-2));

    % Compute error
    e(k) = yzad(k) - y(k);
    e_sum = e_sum + e(k)^2;

    % Compute manipulate variable for discrete time k
    u(k) = r2*e(k-2) + r1*e(k-1) + r0*e(k) + u(k-1);
```

```

    % Constrains on value
    if(u(k) < u_min)
        u(k) = u_min;
    elseif(u(k) > u_max)
        u(k) = u_max;
    end
end
len = length(y);

fprintf("Error sum: %02f \r\n", e_sum);

figure;
stairs(1:len, y);
hold on;

figure;
stairs(1:len, u);

```

1.3.2. DMC

Kod algorytmu DMC w najprostszej wersji analitycznej przedstawiono na listingu 1.5. Został on stworzony na podstawie programu z laboratorium 1.

Listing 1.5: Rozmyty algorytm PID

```

function [y, u, e_sum] = dmc(N, Nu, D, lambda, Upp, step1_value, ...
    step2_value, step3_value, step4_value)
    u_min = -1;
    u_max = 1;
    start = 10;
    kend = 800;

    % Set trajectory
    step1_time = 10;
    step2_time = 200;
    step3_time = 400;
    step4_time = 600;

    yzad(1:step1_time) = 0;
    yzad(step1_time:step2_time) = step1_value;
    yzad(step2_time:step3_time) = step2_value;
    yzad(step3_time:step4_time) = step3_value;
    yzad(step4_time:kend) = step4_value;

    Yodp = stepResponse(0, 0, Upp, D);
    Ypp = Yodp(D);

    % Step response
    s = stepResponseNormalized(Upp, Ypp, 0.01, D+1);
    s = s(2:end);

    % Fill M matrix
    M = zeros(N, Nu);
    for i=1:Nu
        M(i:end,i)=s(1:N-i + 1);
    end

    % Fill MP matrix
    MP = zeros(N, D-1);
    for i = 1:N
        for j = 1:D-1

```

```

        if i+j <= D
            MP(i, j) = s(i+j) - s(j);
        else
            MP(i, j) = s(D) - s(j);
        end
    end
end

% Regulator parameters
I = eye(Nu);
K = ((M'*M+lambda*I)^(-1))*M';
Ku = K(1,:)*MP;
Ke = sum(K(1, :));

% Variables initialization
y = ones(kend, 1) * Ypp;
u = ones(kend, 1) * Upp;
deltauk_p = zeros(D-1, 1);

e_sum = 0;

% Main loop
for k=start:kend
    if k >= 2
        Ykm1 = y(k-1);
    else
        Ykm1 = Ypp;
    end
    if k >= 3
        Ykm2 = y(k-2);
    else
        Ykm2 = Ypp;
    end
    if k >= 6
        Ukm5 = u(k-5);
    else
        Ukm5 = Upp;
    end
    if k >= 7
        Ukm6 = u(k-6);
    else
        Ukm6 = Upp;
    end

    y(k) = symulacja_obiektu11y_p3(Ukm5, Ukm6, Ykm1, Ykm2);

    % Compute error
    ek = yzad(k) - y(k);
    e_sum = e_sum + ek^2;

    % Compute deltau variable for given control horizon
    deltau_k = Ke*ek-Ku*deltauk_p;

    % Back deltau window
    for n=D-1:-1:2
        deltau_k_p(n) = deltau_k_p(n-1);
    end

    % Manipulate variable for time k
    u(k) = u(k-1) + deltau_k;

    if(u(k) < u_min)
        u(k) = u_min;
    end
end

```



```

elseif(u(k) > u_max)
    u(k) = u_max;
end

% Delta u for time k
deltauk_p(1) = u(k) - u(k-1);
end
end

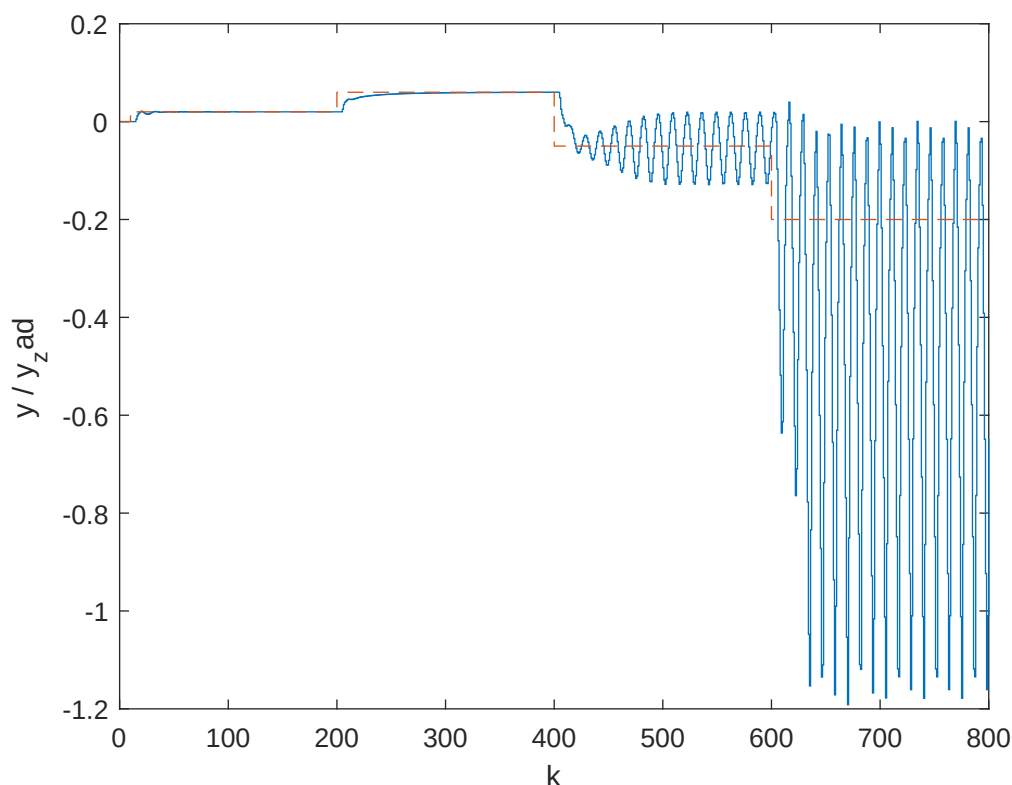
```

1.4. Testy algorytmów PID i DMC na nieliniowym procesie

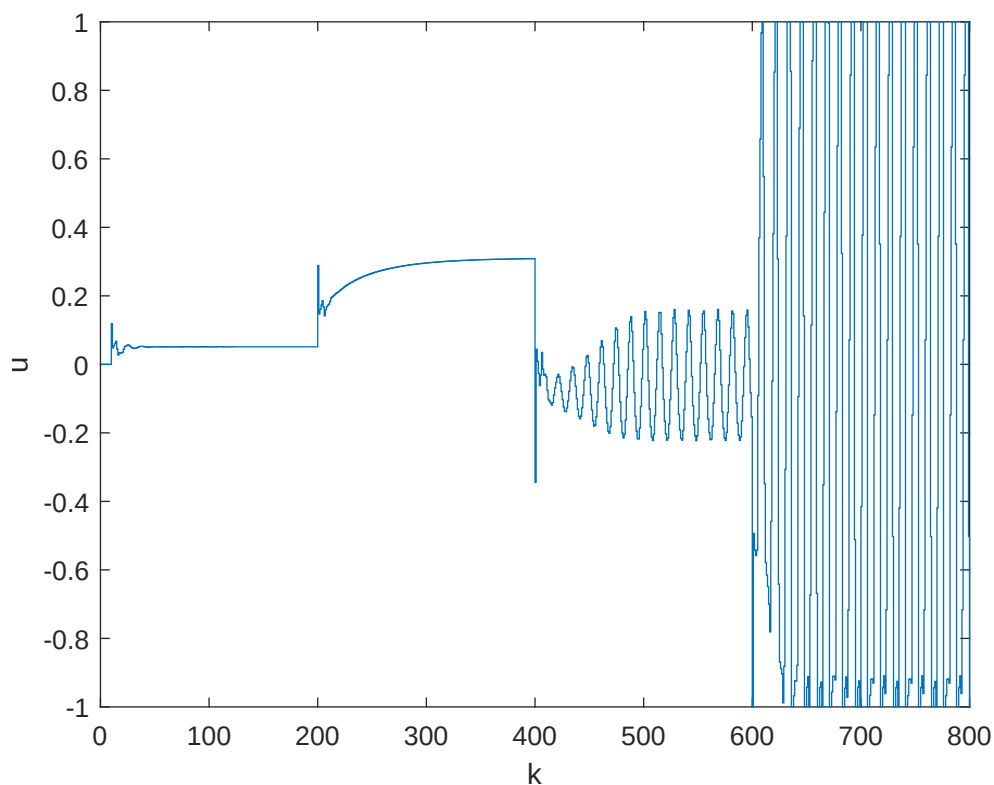
Algorytmy testowano na trajektorii wartości zadanej, która zakładała skoki w chwilach 10, 200, 400 i 600 do wartości kolejno 0.02, 0.06, -0.05, -0.2. Wartości dobrano tak, aby przetestować działanie regulatorów w szerokim zakresie wartości zadanej.

1.4.1. PID

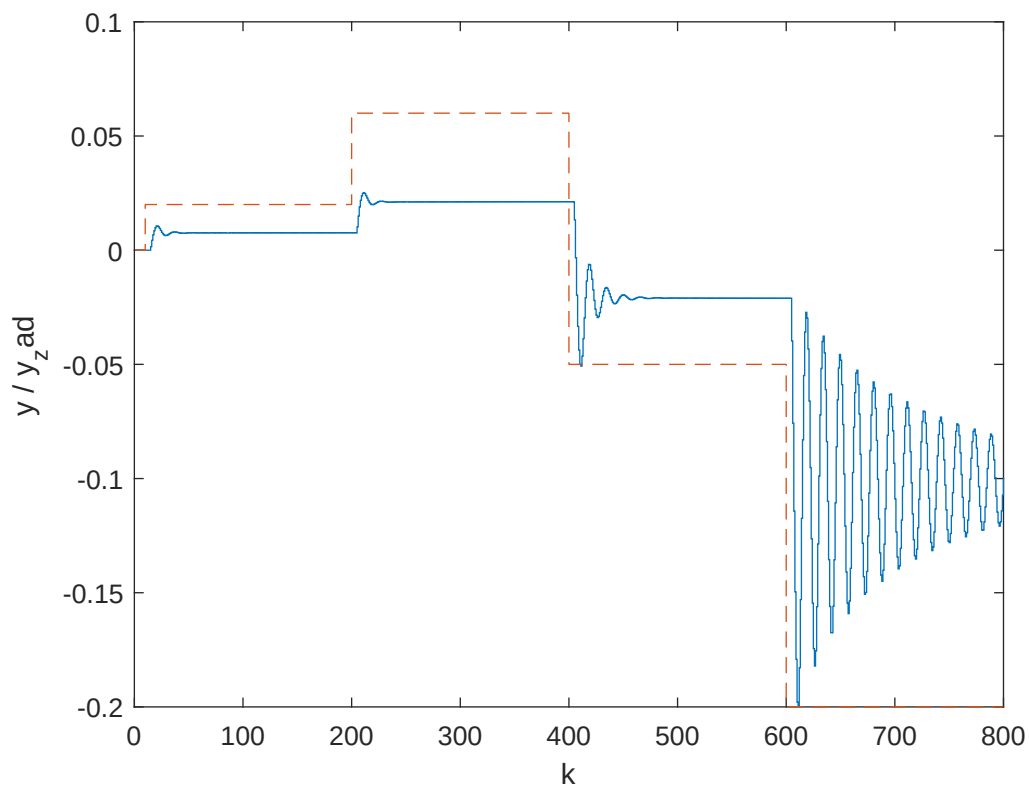
Pierwszym podejściem do strojenia regulatora PID było wystrojenie go dobrze do wartości pierwszego skoku wartości zadanej. Zastosowano do tego celu metodę inżynierską. Wyznaczona wartość K_u wyniosła 3.85, więc $K_p = 0.5K_u = 1.925$. Nastawy $T_i = 3$, $T_d = 1$ dobrano eksperymentalnie obserwując błąd średniokwadratowy. Gdy wyznaczone nastawy zastosowano do całej trajektorii wartości zadanej, można zobaczyć, że jakość regulacji PID okazała się bardzo słaba. Dla skoków ujemnych obiekt wpada w oscylacje. Nic dziwnego, obiekt jest silnie nieliniowy. Rezultaty eksperymentu pokazano na rysunkach 1.3 i 1.4.



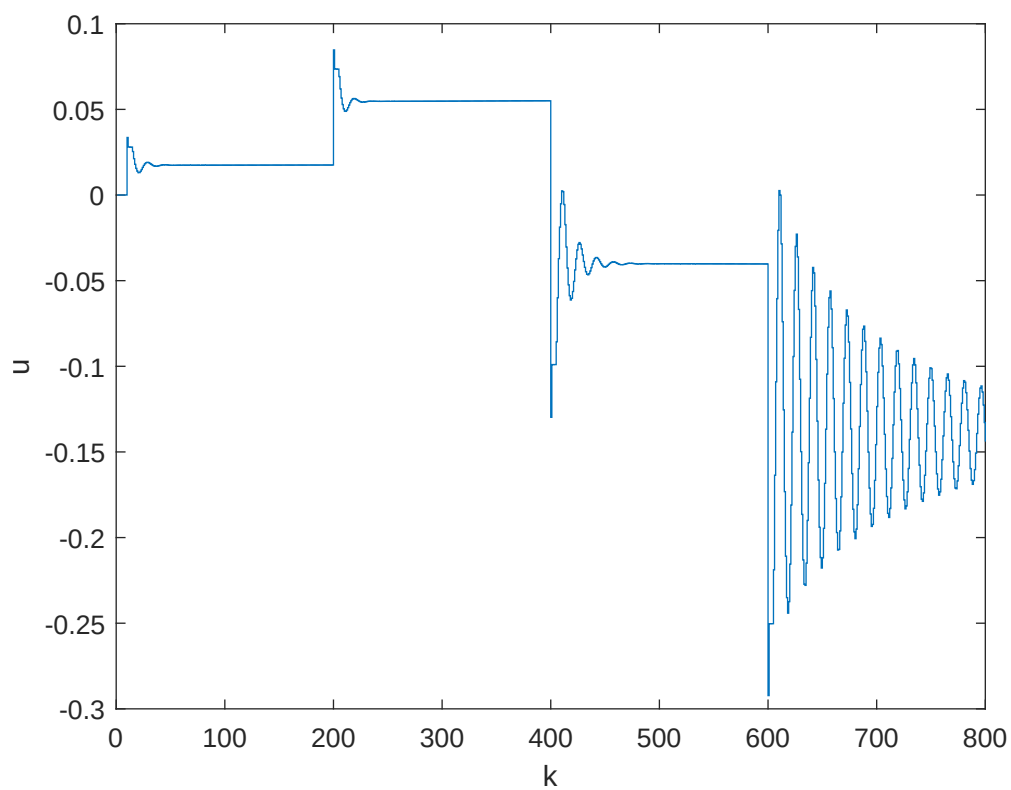
Rys. 1.3: Sygnał wyjściowy PID, $K_p = 1.925$, $T_i = 3$, $T_d = 1$. Błąd $E = 42.37$

Rys. 1.4: Sterowanie PID, $K_p = 1.925, T_i = 3, T_d = 1$

W toku dalszych eksperymentów okazało się, że najmniejszy błąd średniokwadratowy dają nastawy $K_p = 1.4, T_i = 10000, T_d = 0.1$. Ciężko jednak nazwać w tym przypadku działanie regulatora regulacją - w przypadku żadnego skoku wartości zadanej sygnał wyjściowy nie dochodzi do wartości zadanej (z powodu wyłączzonego członu I, jednak gdy go włączymy błąd średniokwadratowy wzrasta z powodu większych oscylacji). Rezultaty eksperymentu pokazano na rysunkach 1.5 i 1.6.



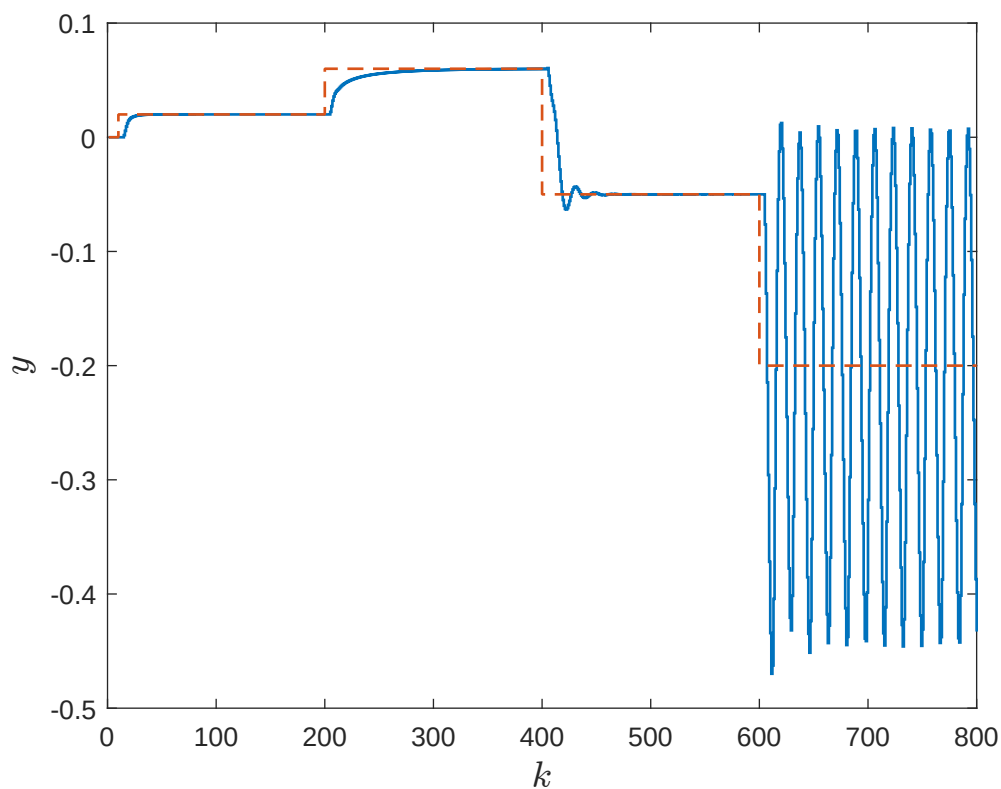
Rys. 1.5: Sygnał wyjściowy PID, $K_p = 1.4, T_i = 10000, T_d = 0.1$. Błąd $E = 2.76$



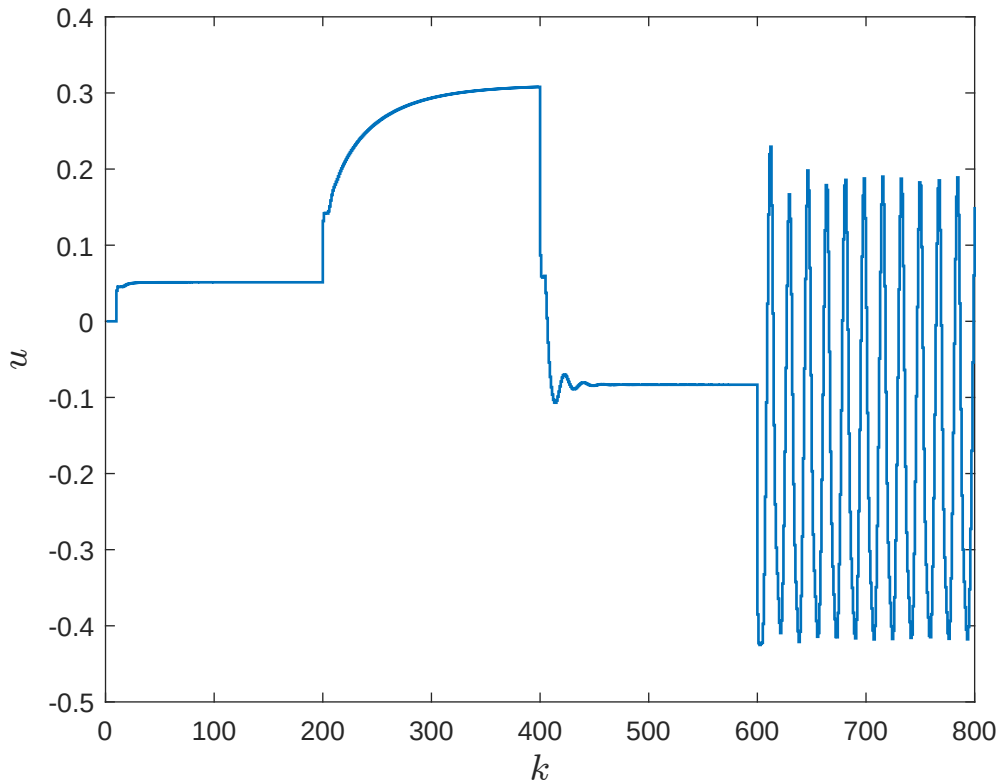
Rys. 1.6: Sterowanie PID, $K_p = 1.4, T_i = 10000, T_d = 0.1$

1.4.2. DMC

Klasyczny regulator DMC, na którym przeprowadzono test ma swój punkt pracy w okolicach $(U_{pp}, Y_{pp}) = (0, 0)$. Na rysunku 1.7 widzimy, że dla wartości zadanej nie odbiegającej bardzo dużo od 0 regulator radzi sobie nieźle. Jednak dla $y_{zad} = -0.2$ algorytm DMC wpada w niegasnące oscylacje. Tym regulatorem również nie uda się nam wyregulować procesu. Przebieg sygnału sterującego zamieszczono na rysunku 1.8.



Rys. 1.7: Sygnał wyjściowy DMC, $D = 50, N = 50, N_u = 1, \lambda = 1$. Błąd $E = 5.21$



Rys. 1.8: Sterowanie DMC, $D = 50$, $N = 50$, $N_u = 1$, $\lambda = 1$. Błąd $E = 5.21$

1.5. Rozmyty PID i rozmyty DMC

W tej sekcji znajdują się kody rozmytych algorytmów PID i DMC.

1.5.1. Funkcja przynależności

Aby zaimplementować regulatory rozmyte potrzebujemy określić funkcję przynależności. W przypadku tego projektu wybrano trapezową funkcję przynależności z powodu jej prostoty i skuteczności.

Zaimplementowano funkcję **membershipFunction** (listing 1.6). Na ujemnym brzegu dozwolonych wartości u przyjmuje ona wartość 1 dla regulatora lokalnego o punkcie pracy pierwszym w kolejności a 0 dla pozostałych regulatorów. Za tym punktem przez $\frac{1}{4}$ dystansu pomiędzy kolejnymi punktami pracy nic się nie zmienia. Pomiędzy $\frac{1}{4}$ a $\frac{3}{4}$ dystansu znajduje się właściwa część rozmyta, aby po $\frac{3}{4}$ dystansu między punktami pracy przejść w wartość 1 dla regulatora o drugim punkcie pracy, a 0 dla reszty. Na dodatnim brzegu dozwolonych wartości u zachodzi podobna sytuacja jak na ujemnym, tylko że wartość przynależności 1 ma regulator o najbardziej dodatnim punkcie pracy, a reszta regulatorów 0.

Wygląd funkcji przynależności dla przykładowych punktów pracy -0.8, 0, 0.4 przedstawiono na rysunku 1.9.

Listing 1.6: Funkcja membershipFunction

```
function [values] = membershipFunction(working_points, u)
    % Constraint: values in working_points vector should be greater
    % than u_min, smaller than u_max and be in the ascending order

    num_regulators = length(working_points);
    values = zeros(num_regulators, 1);
```

```

if num_regulators == 1
    values = 1;
    return;
end

for i=1:num_regulators-1
    working_point_1 = working_points(i);
    working_point_2 = working_points(i+1);

    if u <= working_point_2
        distance_1_2 = working_point_2 - working_point_1;
        if u <= working_point_1 + distance_1_2 * 0.25
            values(i) = 1;
            return;
        elseif u >= working_point_2 - distance_1_2 * 0.25
            values(i+1) = 1;
            return;
        else
            values(i) = (working_point_2 - distance_1_2 * 0.25 - u) ...
                / (distance_1_2 * 0.5);
            values(i+1) = 1 - values(i);
            return;
        end
    end
end

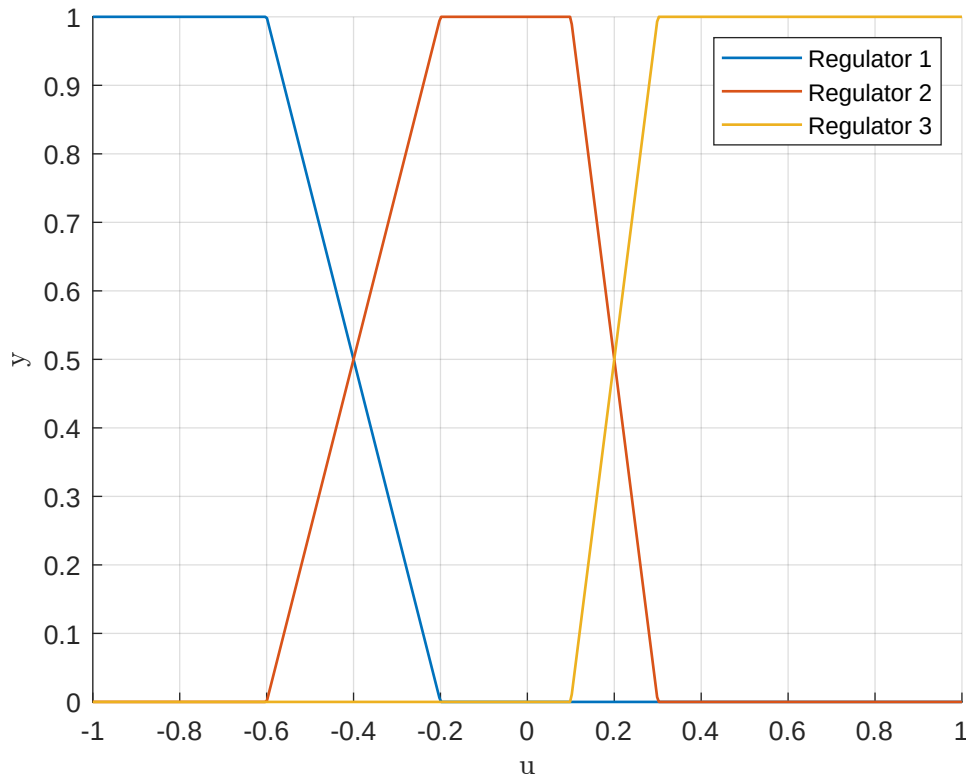
if u >= working_points(num_regulators)
    values(num_regulators) = 1;
end
end

```

Następnie sterowania regulatora rozmytego są obliczane ze wzoru:

$$u(k) = \frac{\sum_{j=1}^r w^j(k) u^j(k)}{\sum_{l=1}^r w^l(k)}$$

Gdzie w_j to poziomy aktywacji poszczególnych reguł regulatora, a u_i to wyjścia regulatorów lokalnych. W przypadku stosowanej w projekcie funkcji przynależności mianownik tego wyrażenia zawsze wynosi 1.



Rys. 1.9: Funkcja przynależności dla przykładowych punktów pracy -0.8, 0, 0.4

1.5.2. Rozmyty PID

Aby zaimplementować rozmyty algorytm PID stworzono funkcję **fuzzyPID**, która przyjmuje nastawy (jako wektory o długości równej ilości regulatorów lokalnych), numery iteracji początku i startu, trajektorię wartości zadanej oraz wektor punktów pracy regulatorów lokalnych. Długość tego wektora jest liczbą lokalnych regulatorów.

Prawo regulacji jakie zastosowano to:

$$u(k) = u_P(k) + u_I(k) + u_D(k)$$

gdzie:

$$u_P(k) = K_p e(k)$$

$$u_I(k) = u_I(k-1) + \frac{K_p T}{T_I} \frac{e(k-1) + e(k)}{2}$$

$$u_D(k) = K_p T_D \frac{e(k) - e(k-1)}{T}$$

Ponadto wprowadzono ograniczenia sygnału sterującego zgodne z ograniczeniami symulowanego procesu. Do rozmywania korzystamy z pokazanej wcześniej funkcji przynależności `membershipFunction`. Kod znajduje się na listingu 1.6.

Listing 1.7: Rozmyty algorytm PID

```
function [y, u, e_sum] = fuzzyPID(Kp, Ti, Td, start, kend, working_points, yzad)
    u = zeros(1, kend); y = zeros(1, kend);
    e = zeros(1, kend);
    up = zeros(1, kend); ui = zeros(1, kend); ud = zeros(1, kend);

    u_min = -1;
    u_max = 1;
```

```

e_sum = 0;
num_regulators = length(working_points);

for k=start:kend
    y(k) = symulacja_obiektu11y_p3(u(k-5), u(k-6), y(k-1), y(k-2));

    % Compute error
    e(k) = yzad(k) - y(k);
    e_sum = e_sum + e(k)^2;

    membership_weights = membershipFunction(working_points, u(k-1));

    for i=1:num_regulators
        if membership_weights(i) == 0
            continue;
        end

        up(k) = Kp(i) * e(k);
        ui(k) = ui(k-1) + Kp(i) * (e(k-1) + e(k)) / (2 * Ti(i));
        ud(k) = Kp(i) * Td(i) * (e(k) - e(k-1));
        u_reg = up(k) + ui(k) + ud(k);

        u(k) = u(k) + membership_weights(i) * u_reg;
    end

    if(u(k) < u_min)
        u(k) = u_min;
    elseif(u(k) > u_max)
        u(k) = u_max;
    end
end
end

```

1.5.3. Rozmyty DMC

Do celów implementacji rozmytego regulatora DMC zaimplementowano klasę lokalnego DMC o nazwie **localDMC** (listing 1.8).

Obliczenia offline są wykonywane w konstruktorze, obliczenia online w funkcji **iterationDMC**. Po każdej iteracji trzeba wywołać odświeżenie wektora przeszłych sterowań za pomocą funkcji **updateAfterIteration**. Sam sposób implementacji (wykonywania obliczeń, nazewnictwa macierzy i wektorów) jest zaczerpnięty z implementacji regulatora DMC z listingu 1.5.

Listing 1.8: Lokalny regulator DMC

```

classdef localDMC
    properties
        % Constraints
        u_min = -1
        u_max = 1
        % Trajectory
        yzad
        % Regulator parameters
        deltauk_p
        u
        Ke
        Ku
        % Horizont
        D
    end

    methods

```



```

function obj = localDMC(N, Nu, D, lambda, Upp, yzad, kend)
    obj.D = D;
    obj.yzad = yzad;

    Yodp = stepResponse(0, 0, Upp, 50);
    Ypp = Yodp(50);

    % Step response
    s = stepResponseNormalized(Upp, Ypp, 0.01, D+1);
    s = s(2:end);

    % Fill M matrix
    M = zeros(N, Nu);
    for i=1:Nu
        M(i:end,i)=s(1:N-i + 1);
    end

    % Fill MP matrix
    MP = zeros(N, D-1);
    for i = 1:N
        for j = 1:D-1
            if i+j <= D
                MP(i, j) = s(i+j) - s(j);
            else
                MP(i, j) = s(D) - s(j);
            end
        end
    end

    % Regulator parameters
    I = eye(Nu);
    K = ((M'*M+lambda*I)^(-1))*M';
    obj.Ku = K(1,:)*MP;
    obj.Ke = sum(K(1, :));

    % Variables initialization
    obj.u = zeros(kend, 1);
    obj.deltauk_p = zeros(D-1, 1);
end

function u_current = iterationDMC(obj, k, y_current)
    % Compute error
    ek = obj.yzad(k) - y_current;

    % Compute deltau variable for given control horizon
    deltauk = obj.Ke * ek - obj.Ku * obj.deltauk_p;

    % Manipulate variable for time k
    u_current = obj.u(k-1) + deltauk;

    if(u_current < obj.u_min)
        u_current = obj.u_min;
    elseif(u_current > obj.u_max)
        u_current = obj.u_max;
    end
end

function obj = updateAfterIteration(obj, last_u, k)
    % Back deltau window
    for n=obj.D-1:-1:2
        obj.deltauk_p(n) = obj.deltauk_p(n-1);
    end
end

```

```

        obj.u(k) = last_u;
        obj.deltauk_p(1) = obj.u(k) - obj.u(k-1);
    end
end
end

```

Właściwa implementacja rozmytego regulatora DMC znajduje się na listingu 1.9. Funkcja **fuzzyDMC** przyjmuje wektory N , N_u i λ o długości równej liczbie regulatorów lokalnych, wartość D , trajektorię zadaną i wektor punktów pracy regulatorów lokalnych. Najpierw inicjalizujemy regulatory lokalne DMC i zapisujemy do wektora **regulators**. Później w pętli obliczamy wartość sterowania na podstawie wyjść regulatorów lokalnych i funkcji przynależności. Po każdej iteracji należy zaktualizować wektor przeszłych sterowań regulatorów lokalnych.

Listing 1.9: Rozmyty algorytm DMC

```

function [y, u, e_sum] = fuzzyDMC(N, Nu, D, lambda, working_points, yzad)
    start = 10;
    kend = 800;

    y = zeros(kend, 1);
    u = zeros(kend, 1);
    e_sum = 0;

    num_regulators = length(working_points);
    regulators = [];

    % Initialize local regulators
    for i=1:num_regulators
        regulators = [regulators localDMC(N(i), Nu(i), D, lambda(i), ...
            working_points(i), yzad, kend)];
    end

    % Main loop
    for k=start:kend
        y(k) = symulacja_obiektu11y_p3(u(k-5), u(k-6), y(k-1), y(k-2));

        % Compute error
        ek = yzad(k) - y(k);
        e_sum = e_sum + ek^2;

        membership_weights = membershipFunction(working_points, u(k-1));

        % Get u from local regulators
        for i=1:num_regulators
            if membership_weights(i) == 0
                continue;
            end

            u_reg = regulators(i).iterationDMC(k, y(k));
            u(k) = u(k) + membership_weights(i) * u_reg;
        end

        % Update local regulators
        for i=1:num_regulators
            regulators(i) = regulators(i).updateAfterIteration(u(k), k);
        end
    end
end

```

1.6. Testy rozmytych algorytmów PID i DMC

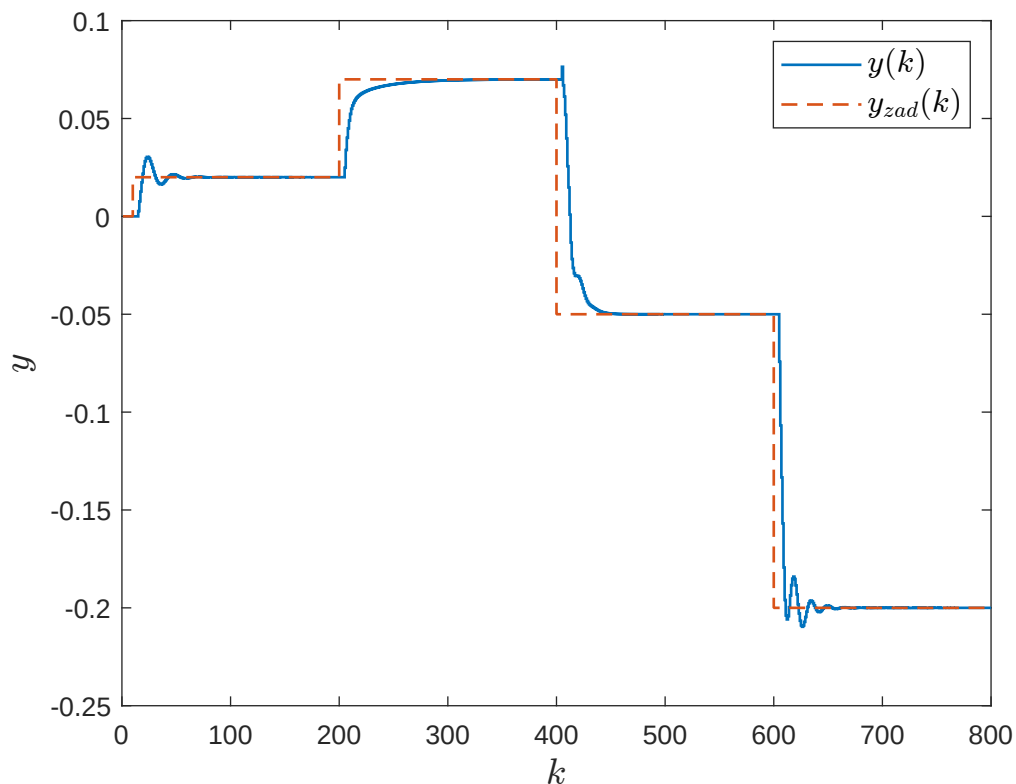
Testy prowadzono dla trajektorii wartości zadanej takiej samej jak dla testów klasycznych algorytmów PID i DMC.

1.6.1. Rozmyty PID

Rozpoczęto od dwóch regulatorów lokalnych. Na początku dobrano nastawy metodą inżynierską dla dwóch skoków wartości zadanej: do 0.02 i do -0.2 (dwa oddzielne eksperymenty). Uzyskano rezultaty: dla 0.02 $K_p = 1.925$, $T_i = 3$, $T_d = 1$, dla -0.2 $K_p = 0.6$, $T_i = 3$, $T_d = 1$.

Dalsze eksperymenty prowadzono już na regulatorze rozmytym. Jako punkty pracy wybrano takie u , jakie mniej więcej było osiągane w stanie stabilnym dla wartości zadanych, na jakich przeprowadzano dwa powyższe eksperymenty, czyli -0.2 i 0.1. W toku eksperymentów okazało się, że korzystniej dla regulacji jest dla pierwszego regulatora lokalnego zwiększyć wartość T_i na 5 i T_d na 2. Ponadto punkty pracy regulatorów lokalnych zmieniono na -0.1, 0.1.

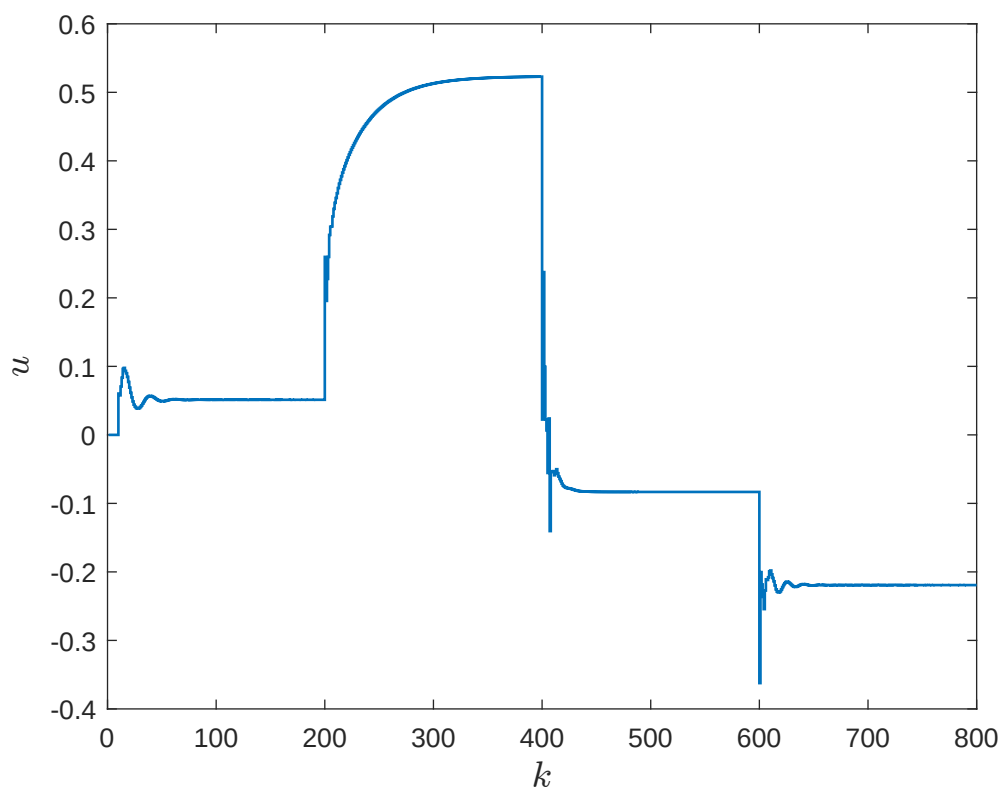
Ostateczne nastawy lokalnych regulatorów PID przedstawiono w tabeli 1.1, wykres wyjścia procesu na rysunku 1.10 a sterowania na rysunku 1.11.



Rys. 1.10: Sygnał wyjściowy, rozmyty PID, 2 regulatory lokalne (-0.1, 0.1), błąd $E = 0.320$

Nr regulatora	U_{pp}	K_p	T_i	T_d
1	-0.1	0.6	5	2
2	0.1	1.925	3	1

Tab. 1.1: Nastawy 2 lokalnych regulatorów PID



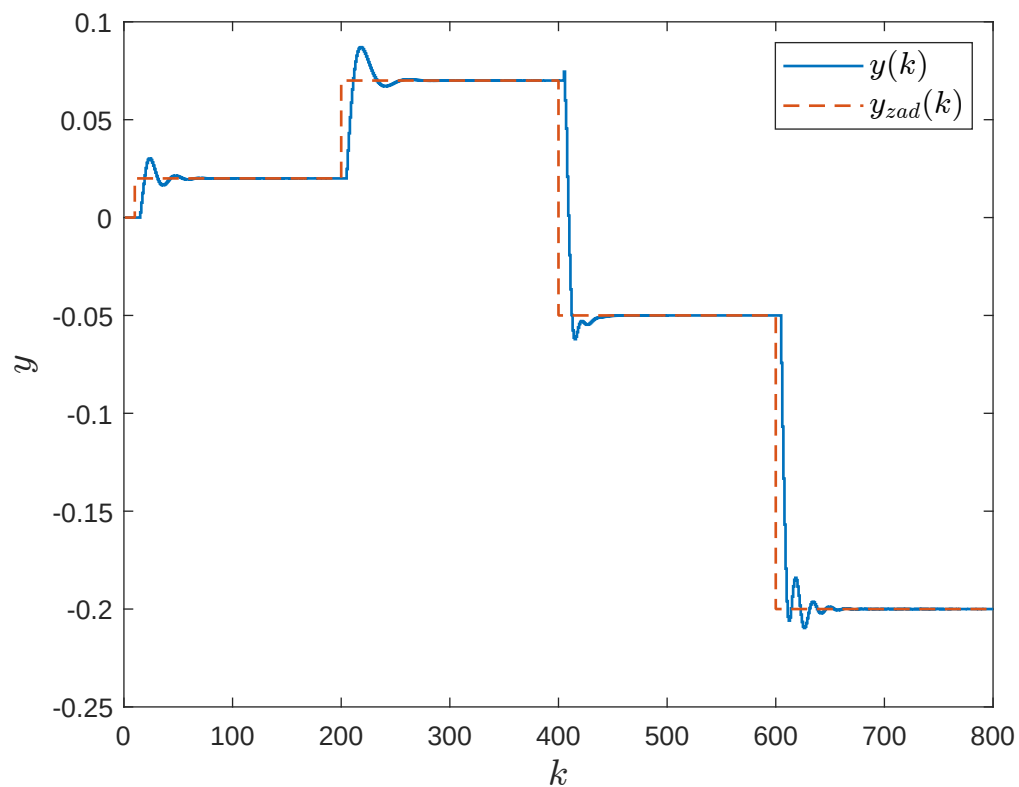
Rys. 1.11: Sterowanie, rozmyty PID, 2 regulatory lokalne (-0.1, 0.1)

Dla 3 regulatorów lokalnych udało się dobrać nastawy, które dodatkowo poprawiają jakość regulacji. Nastawy i punkty pracy dwóch pierwszych regulatorów lokalnych pozostają bez zmian, dołączono trzeci lokalny PID o punkcie pracy 0.3 i $K_p = 2.5$, $T_i = 1$, $T_d = 1$ (przedstawiono to w tabeli 1.2). Udało się zmniejszyć błąd średniokwadratowy do 0.287. Przebiegi wyjścia procesu i sygnału sterującego przedstawiono na rysunkach 1.12 i 1.13.

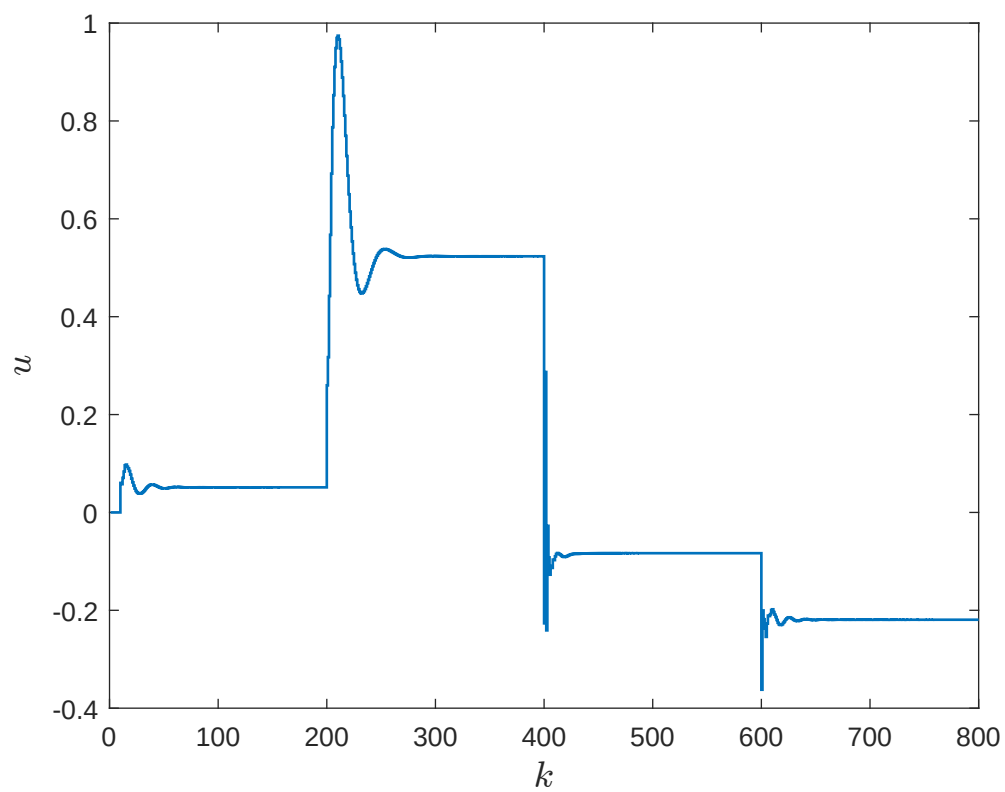
Eksperymenty dla 4 lokalnych regulatorów PID okazały się nieudane (nie przyniosły lepszych rezultatów), przestano więc na 3 regulatorach lokalnych.

Nr regulatora	U_{pp}	K_p	T_i	T_d
1	-0.1	0.6	5	2
2	0.1	1.925	3	1
3	0.3	2.5	1	1

Tab. 1.2: Nastawy 3 lokalnych regulatorów PID



Rys. 1.12: Sygnał wyjściowy, rozmyty PID, 3 regulatory lokalne (-0.1, 0.1, 0.3), błąd $E = 0.287$

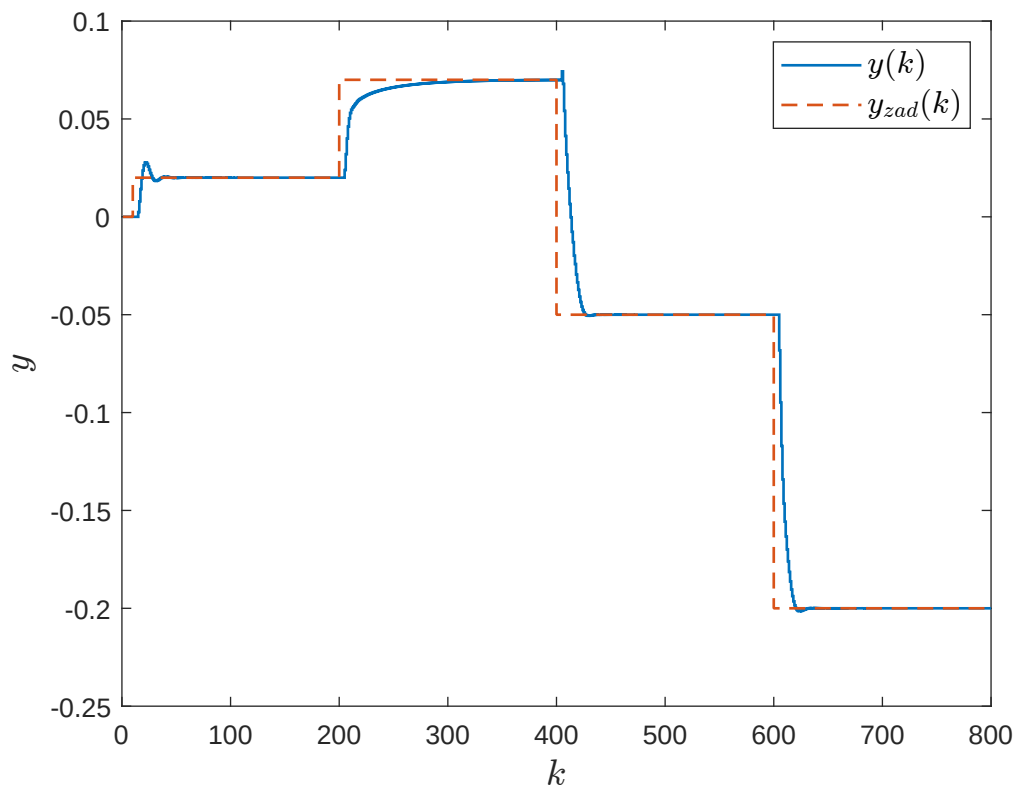


Rys. 1.13: Sterowanie, rozmyty PID, 3 regulatory lokalne (-0.1, 0.1, 0.3)

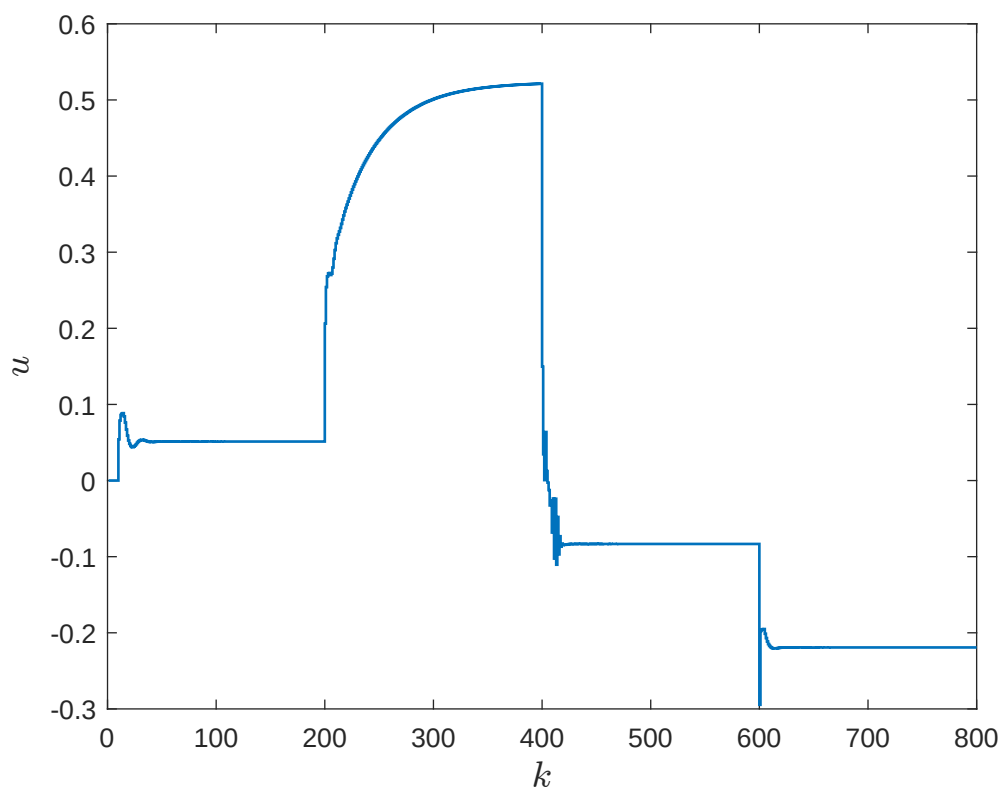
1.6.2. Rozmyty DMC

Na początku testy przeprowadzono dla 2 regulatorów lokalnych. Eksperymentowano z doбором punktów pracy regulatorów lokalnych, ostatecznie najlepsze okazały się punkty -0.2 i 0.1. D ustalono na 50. Dla obu regulatorów lokalnych wybrano $N = 50$, $N_u = 1$, $\lambda = 1$. Próbowano eksperymentować z różnymi wartościami horyzontów predykcji i sterowania dla różnych regulatorów, jednak nie przynosiło to poprawy regulacji, więc zaniechano takich eksperymentów. Wykresy na rysunkach 1.14 i 1.15.

Już przy dwóch regulatorach lokalnych jakość regulacji jest dobra. Przy wszystkich skokach wartości zadanej wyjście procesu dochodzi dosyć szybko do wartości zadanej. Mankamentem jest przesterowanie i oscylacje w przypadku pierwszego skoku. Błąd średniokwadratowy wyniósł 0.332. Sygnał sterujący jest dosyć gładki, bez większych skoków (poza chwilą 600).

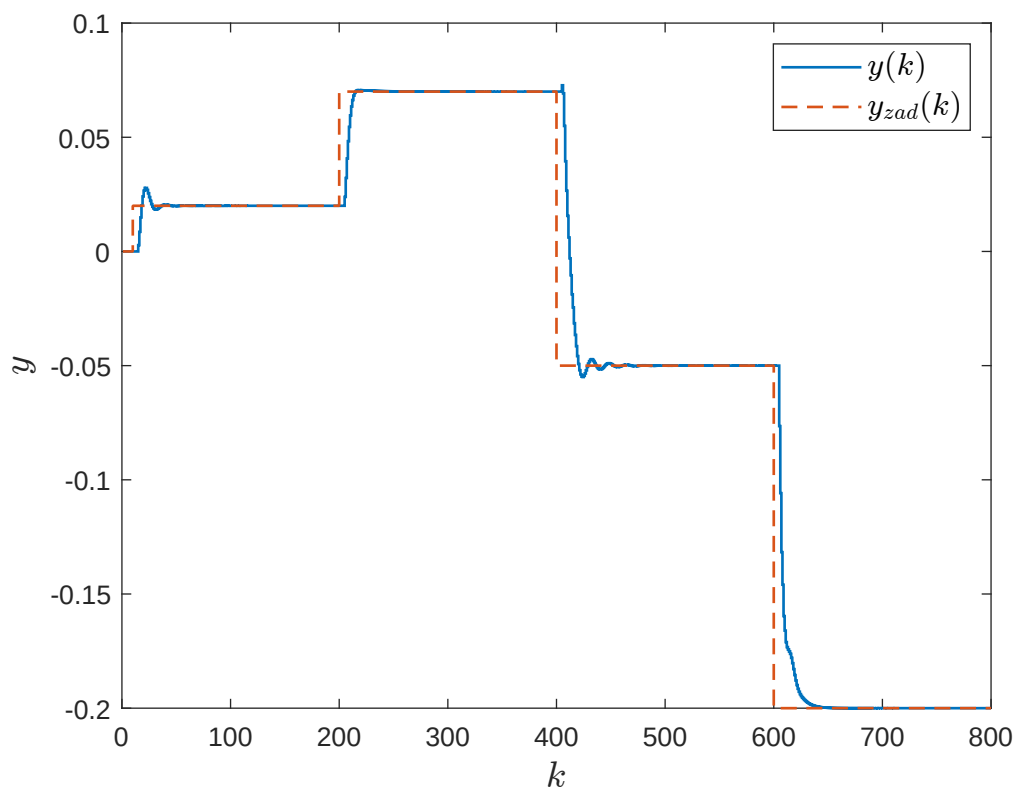


Rys. 1.14: Sygnał wyjściowy, rozmyty DMC, 2 regulatory lokalne (-0.2, 0.1), błąd $E = 0.332$

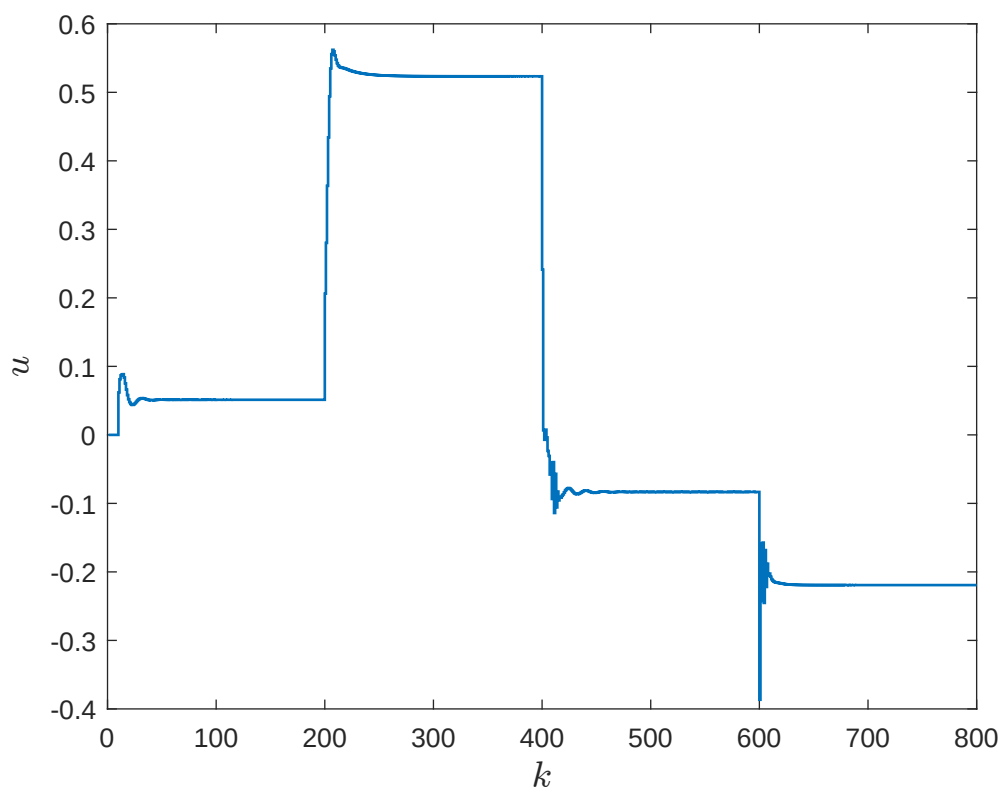


Rys. 1.15: Sterowanie, rozmyty DMC, 2 regulatory lokalne (-0.2, 0.1)

Wprowadzając trzeci regulator lokalny udało się lekko poprawić jakość regulacji. Błąd średniokwadratowy spadł do 0.313. Punkty pracy regulatorów lokalnych dobrano eksperymentalnie na -0.3, 0.1 i 0.3. Wykresy na rysunkach 1.16 i 1.17.

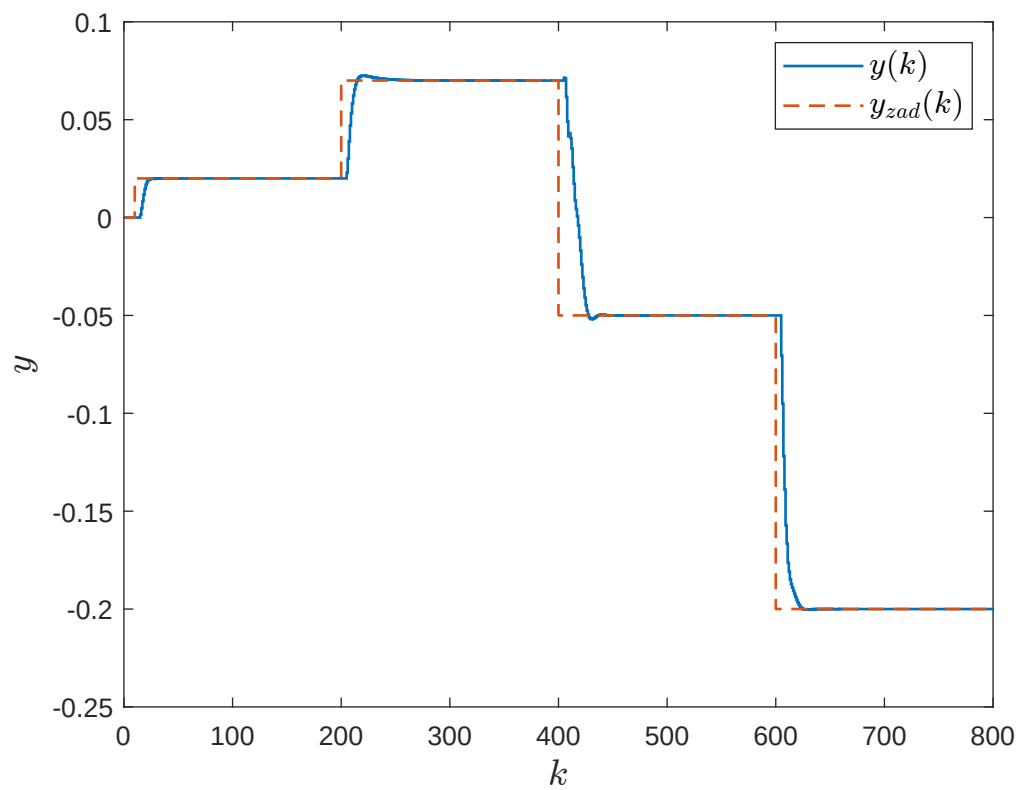


Rys. 1.16: Sygnał wyjściowy, rozmyty DMC, 3 regulatory lokalne (-0.3, 0.1, 0.3), błąd $E = 0.313$

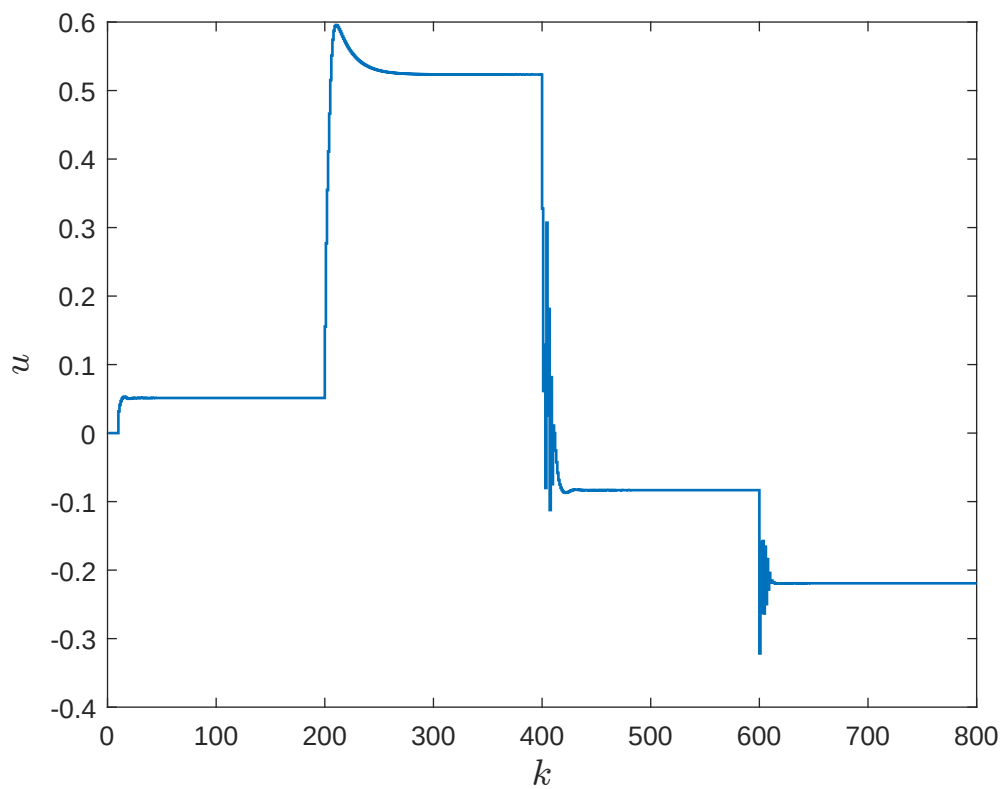


Rys. 1.17: Sterowanie, rozmyty DMC, 3 regulatory lokalne (-0.3, 0.1, 0.3)

Wprowadzeniem czwartego regulatora lokalnego, mimo licznych eksperymentów, nie udało się zmniejszyć błędu średniokwadratowego. Udało się jednak znaleźć takie punkty pracy regulatorów lokalnych, że przy każdym skoku wyjście procesu dochodziło do wartości zadanej praktycznie bez przesterowania i oscylacji. Wykresy na rysunkach 1.18 i 1.19.



Rys. 1.18: Sygnał wyjściowy, rozmyty DMC, 4 regulatory lokalne (-0.3, -0.05, 0.2, 0.5), błąd $E = 0.357$



Rys. 1.19: Sterowanie, rozmyty DMC, 4 regulatory lokalne (-0.3, -0.05, 0.2, 0.5)

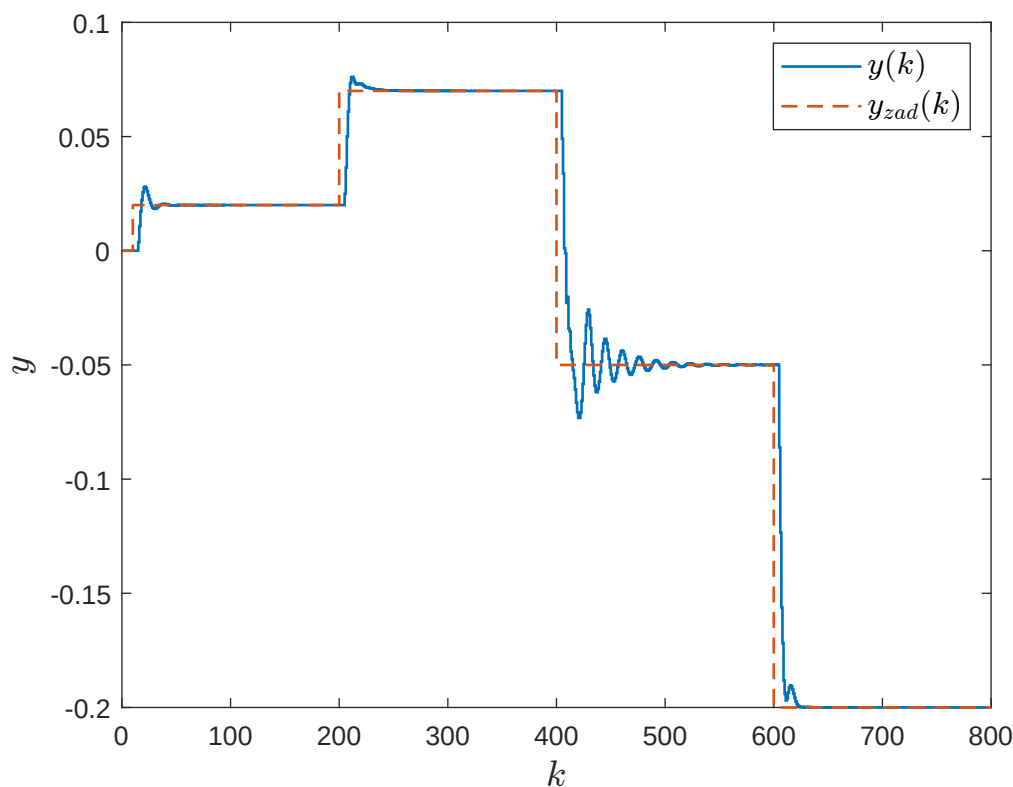
Wprowadzenie piątego regulatora lokalnego, mimo eksperymentów, nie przyniosło lepszych rezultatów. Poprzestano więc na 4 regulatorach lokalnych.

Parametr lambda

Ostatnim etapem testów rozmytego regulatora DMC jest zmniejszenie wartości λ ze stosowanej wcześniej wartości 1. Pozostałe parametry pozostają bez zmian. Parametr λ jest ustawiany w eksperymentach jednakowo dla wszystkich regulatorów lokalnych, kombinowanie z innymi konfiguracjami nie przyniosło dobrych rezultatów.

Zmniejszenie wartości λ do 0.1 przyniosło poprawę błędu średniokwadratowego w przypadku wszystkich regulatorów rozmytych (z 2, 3 i 4 regulatorami lokalnymi). Regulator jest szybszy, jednak bardziej podatny na wpadanie w oscylacje. Odbywa się to także kosztem gwałtowniejszych zmian sygnału sterującego. Przebiegi wyjścia procesu i sygnału sterującego dla $\lambda = 0.1$ dla regulatora z 3 regulatorami lokalnymi znajdują się na rysunkach 1.20 i 1.21. Dalsze zmniejszanie λ do 0.01 nie poprawia błędu średniokwadratowego.

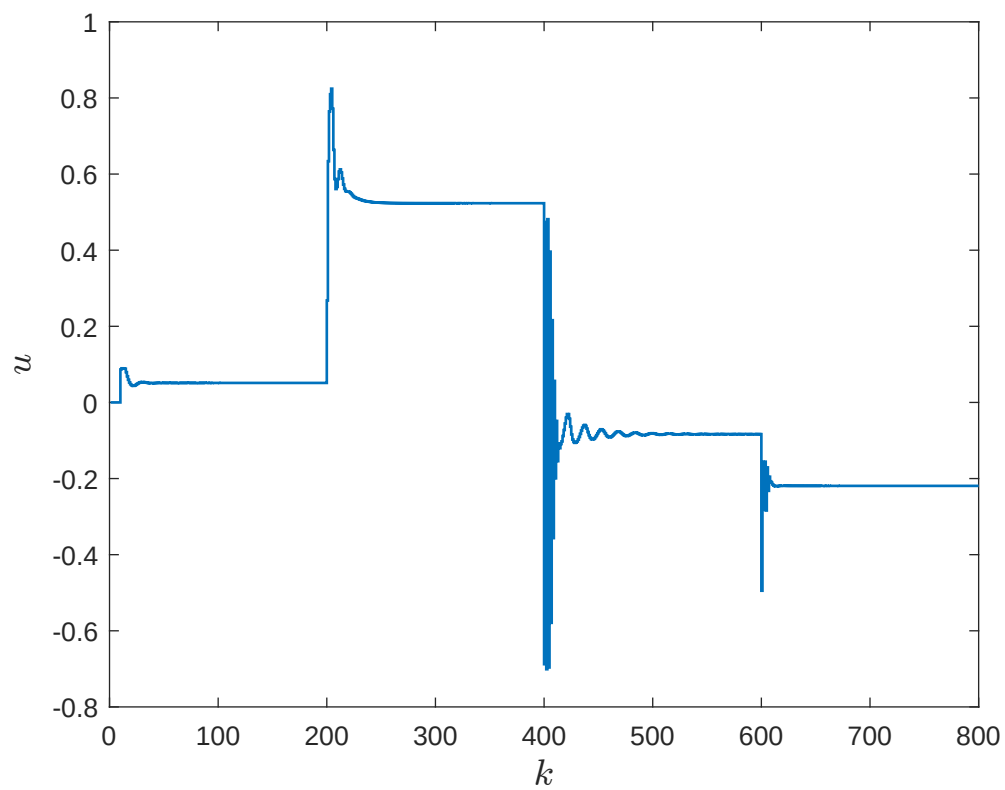
Zwiększenie λ do 10 zwiększa błąd średniokwadratowy, jednak powoduje, że przebieg sygnału sterującego jest bardzo łagodny. Wykresy z taką wartością λ dla regulatora rozmytego o 3 regulatorach lokalnych znajdują się na rysunkach 1.22 i 1.23. Obliczone błędy średniokwadratowe znajdują się w tabeli 1.3.



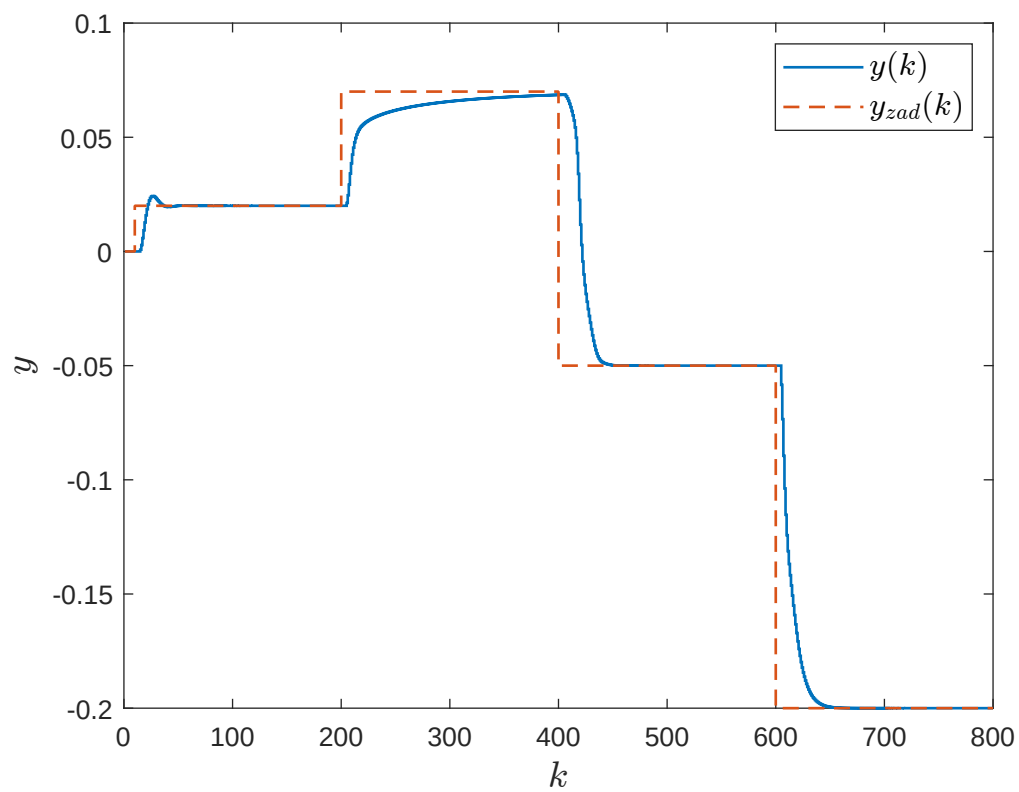
Rys. 1.20: Sygnał wyjściowy, rozmyty DMC, 3 regulatory lokalne, $\lambda = 0.1$, błąd $E = 0.256$

Regulatory lokalne	$\lambda = 10$	$\lambda = 1$	$\lambda = 0.1$	$\lambda = 0.01$
2	0.421	0.332	0.323	0.324
3	0.503	0.312	0.256	0.261
4	0.492	0.357	0.271	0.286

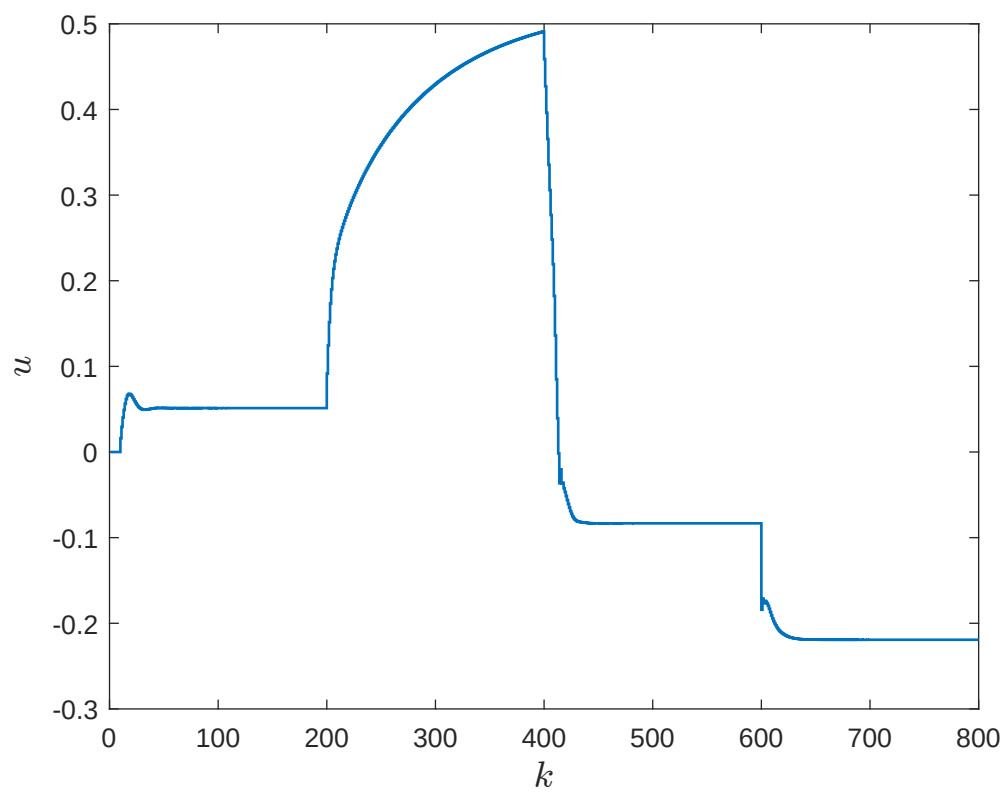
Tab. 1.3



Rys. 1.21: Sterowanie, rozmyty DMC, 3 regulatory lokalne, $\lambda = 10$



Rys. 1.22: Sygnał wyjściowy, rozmyty DMC, 3 regulatory lokalne, $\lambda = 10$, błąd $E = 0.503$



Rys. 1.23: Sterowanie, rozmyty DMC, 3 regulatory lokalne, $\lambda = 10$

2. Laboratorium

2.1. Identyfikacja i modelowanie obiektu

2.1.1. Ustalenie punktu pracy

Pracę na laboratorium rozpoczęto jak zawsze od ustalenia punktu pracy na wyjściu regulowanego przez nas obiektu. Wykorzystano do tego skrypt `heating_station_set_working_point.m`, który został przedstawiony na listingu 2.1.

Listing 2.1: Skrypt umożliwiający ustalenie punktu pracy

```
%% Script used in order to settle working point on the process output

Upp = 26;
FanPower = 50;
Z = 0;
kstart = 1;

%% Communication with heating station setup
addpath('D:\SerialCommunication');
initSerialControl COM4

y = zeros(100, 1);
environment = zeros(100, 1);

%% Configure plot
figure;
hold on;
h_y = stairs(1:kstart, y(1:kstart), 'DisplayName', 'y');
h_environment = stairs(1:kstart, environment(1:kstart), 'DisplayName', ...
    'environment');

title("Wykres sterowania i odpowiedzi stanowiska grzewczego");
xlabel("Iteracje [k]");
ylabel("Wartości");
legend;
grid on;

k = 1;
while(1)
    %% obtaining measurements
    measurements1 = readMeasurements(1); % T1 temperature
    measurements5 = readMeasurements(5); % T5 ambient temperature
    y(k) = measurements1;
    environment(k) = measurements5;

    fprintf('Heater temp: %.2f *C\tEnvironment temp: %.2f *C\n',
        measurements1, measurements5);

    %% sending new values of control signals
    sendControls(1, FanPower);
    sendNonlinearControls(Upp);

    % Update plots
```

```

set(h_y, 'XData', 1:k, 'YData', y(1:k));
set(h_environment, 'XData', 1:k, 'YData', environment(1:k));
drawnow;
k = k + 1;

%% synchronising with the control process
waitForNewIteration();
end

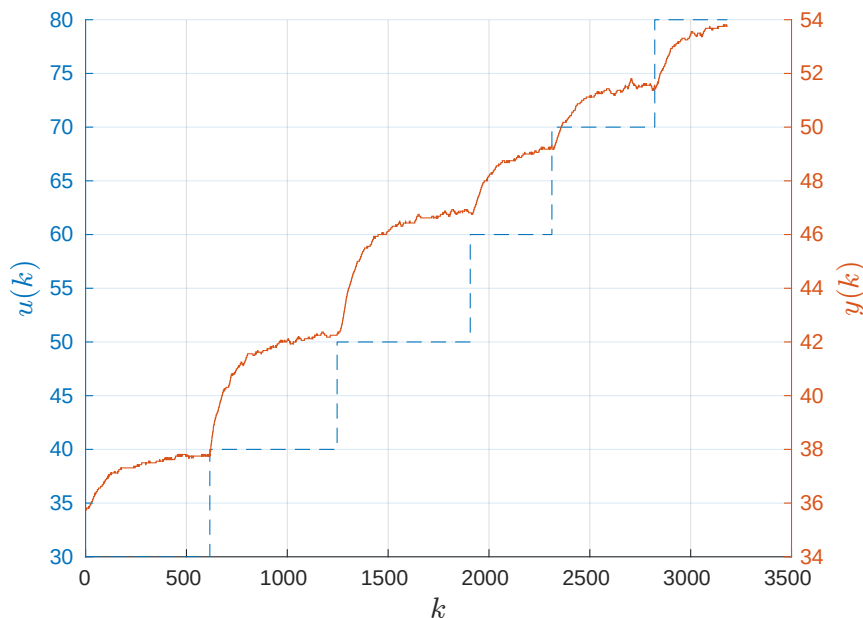
```

Tym razem jednak nie wykorzystano jedynie standardowej funkcji **sendControls**. Sygnał sterujący był przekazywany z użyciem funkcji **sendNonlinearControls**, która to w nieznanym nikomu (oprócz prowadzącego) sposób zmieniała liniowy obiekt w proces nieliniowy.

Po uruchomieniu skryptu ukazanego na listingu 2.1 wyjście obiektu ustabilizowało się na $Y = Y_{pp} = 35,6$.

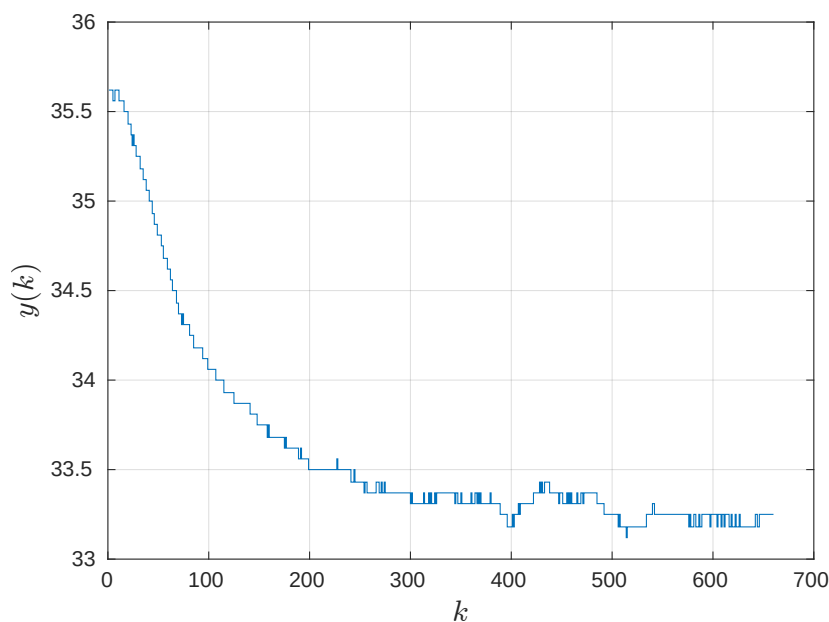
2.1.2. Charakterystyka statyczna

W przypadku tego ćwiczenia, w odróżnieniu od poprzednich, do zamodelowania obiektu potrzebna była większa liczba skoków. Dlatego też zebrano odpowiedzi skokowe dla sygnału sterującego ustalonego na wartościach 20, 30 aż do 80. Co warto uwagi, po każdym skoku nie wracano do początkowego punktu pracy $U_{pp} = 26$, $Y_{pp} = 35,6$, ale każdy następny skok był wykonywany po zakończeniu poprzedniego. Powrót do początkowego punktu pracy nie miałby żadnego znaczenia z punktu widzenia identyfikacji statyki obiektu i niepotrzebnie wydłużyłby i tak już długie pomiary, które trwały ponad 50 minut.



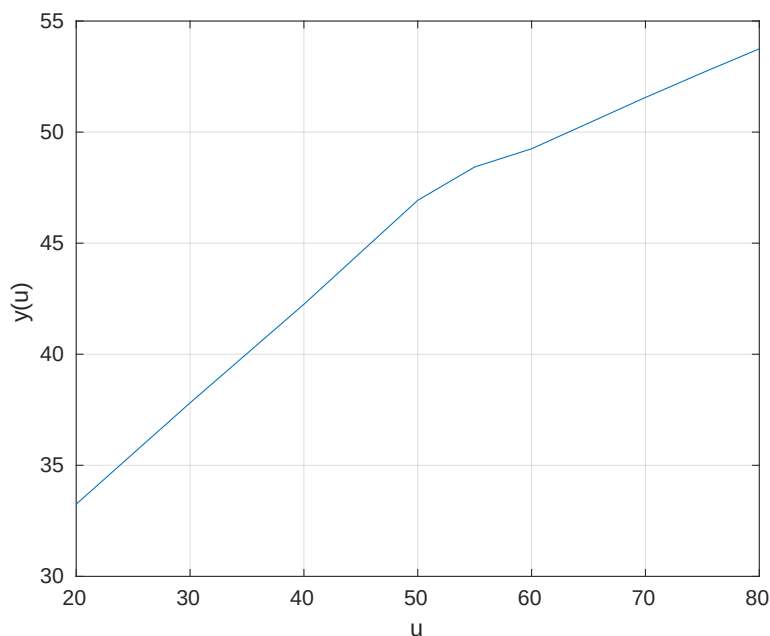
Rys. 2.1: Odpowiedzi skokowe obiektu

Na rys. 2.1 możemy zauważyć, że dla małych wartości sterowania wzmocnienie statyczne obiektu jest większe w porównaniu do sygnałów o większej wartości. Zmiana widoczna jest szczególnie w okolicy $u = 55$.

Rys. 2.2: Odpowiedź skokowa obiektu dla $u = 20$

Skok dla sygnału $u = 20$ został przedstawiony osobno na rys. 2.2 ze względu na to, że był wykonywany oddzielnie i przedstawienie go razem z innymi skokami nie wniosłoby żadnej dodatkowej informacji, a jedynie zaburzyło przekaz.

Dane zebrane podczas wykonywania odpowiedzi skokowych posłużyły do wygenerowania charakterystyki statycznej obiektu, która została przedstawiona na rys. 2.3.

Rys. 2.3: Charakterystyka statyczna $y(u)$ stworzona na podstawie pozyskanych odpowiedzi skokowych

Wnioski wyciągnięte na podstawie wykresu przedstawiającego wszystkie odpowiedzi skokowe, a także treść zadania uzyskały potwierdzenie w charakterystyce statycznej. Po pierwsze, mimo, że nieliniowość nie jest bardzo wyraźna to jest zauważalna. Po drugie punkt przegięcia znajduje się okolicy punktu $u = 55$, tak jak przypuszczano.

Mimo, że obiekt jest nieliniowy to możliwe jest wyliczenie wzmocnienia statycznego dla fragmentów charakterystyki statycznej. Przyjęto, że punkt przełamania jest w $u = 50$.

$$K_{stat} = \begin{cases} 0,45 & \text{dla } u \leq 50 \\ 0,34 & \text{dla } u > 50 \end{cases} \quad (2.1)$$

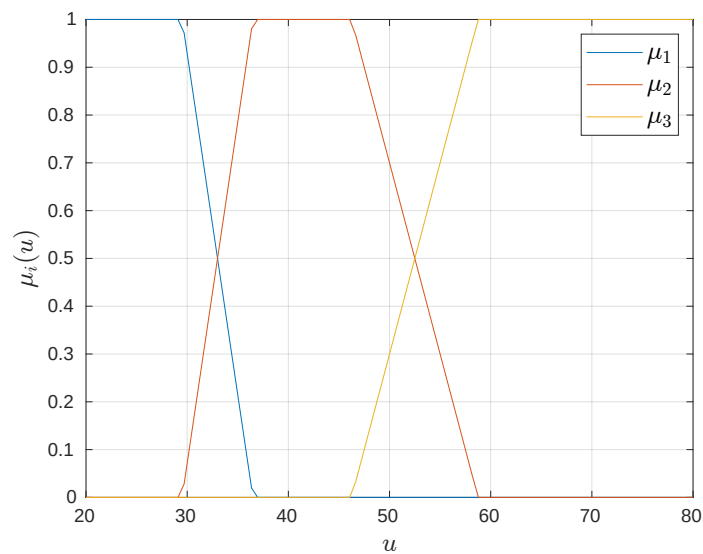
2.1.3. Model rozmyty

Jedną z metod modelowania w przypadku obiektów nieliniowych jest logika rozmyta. W przeciwieństwie do logiki klasycznej przewidującej dwa stany - 0 i 1 lub "Fałsz" i "Prawda" podejście rozmyte przypisuje zmiennej rozmywanej wartości ciągłe w przedziale od 0 do 1.

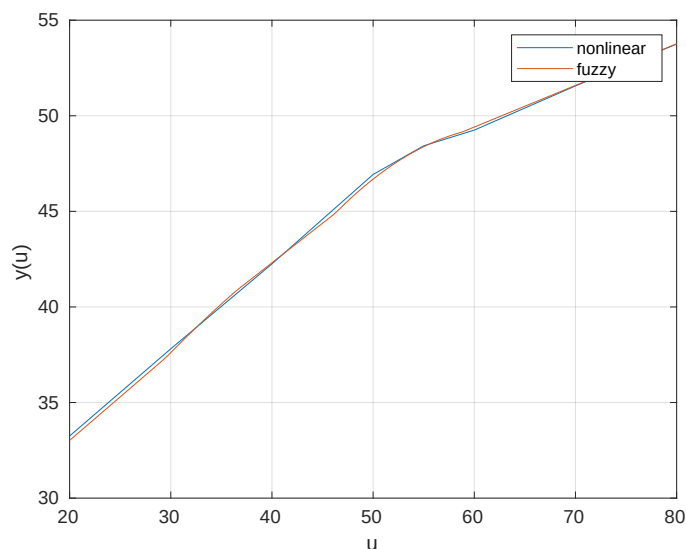
Celem laboratorium było zaprojektowanie rozmytych regulatorów FPID i FDMC zgodnie z podejściem PDC. Każdy z nich miał składać się z 3 regulatorów lokalnych, dlatego też model rozmyty też musiał składać się z 3 modeli lokalnych. Do stworzenia modelu typu Takagi-Sugeno wybrano 3 lokalne punkty pracy $U_{pp1} = 26$, $U_{pp2} = 40$ oraz $U_{pp3} = 65$, z których wykonano skoki do wartości następujące skoki:

$$U_{pp1} = 26 \rightarrow 30 \quad U_{pp2} = 40 \rightarrow 50 \quad U_{pp3} = 65 \rightarrow 75$$

Do tak wybranych punktów pracy dobrano 3 trapezoidalne funkcje przynależności, które przedstawione zostały na rys. 2.4.



Rys. 2.4: Trapezowe funkcje przynależności wykorzystane w modelu rozmytym typu Takagi-Sugeno



Rys. 2.5: Model rozmyty obiektu na tle nieliniowej charakterystyki statycznej badanego procesu

Na rys. 2.5 przedstawiono charakterystykę statyczną modelu rozmytego na tle wyznaczonej charakterystyki statycznej obiektu. Jak widać, już zastosowanie 3 modeli lokalnych doprowadziło do uzyskania bardzo dobrego modelu. Różnice, które są zauważalne nie wynikają z źle dobranych lokalnych punktów pracy czy funkcji przynależności, ale z niedoskonałości, którymi obarczony jest każdy pomiar, szczególnie w tak krótkim czasie.

2.2. Porównanie regulatorów rozmytych z konwencjonalnymi

Regulatory konwencjonalne i rozmyte testowane były na tej samej trajektorii składającej się z dwóch skoków do wartości $Y_{pp} + 5$ oraz $Y_{pp} + 15$. Trajektoria zaczynała się i kończyła na wartości Y_{pp} . Wszystkie wyliczane miary jakości w postaci sumy kwadratu błędów były wyliczane od 0 do 600 iteracji, mimo że niekiedy pokazany wykres jest dłuższy, co wynika z różnych czasów wykonywania poszczególnych eksperymentów.

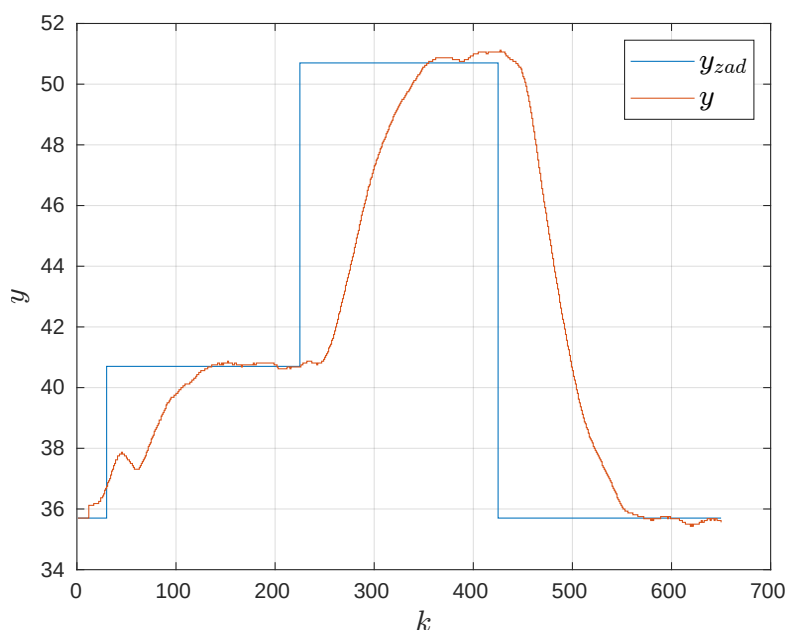
2.2.1. Konwencjonalne regulatory PID i DMC

Przed testami regulatorów rozmytych zaprojektowanych na potrzeby laboratorium przetestowano na "nowym" obiekcie konwencjonalne regulatory liniowe napisane i dostrojone na laboratorium nr 1. Jedyną różnicą był fakt zastosowania nowej funkcji **sendNonlinearControls** zamiast standardowej **sendControls**. Z tego powodu autorzy niniejszego sprawozdania pozwolili sobie nie przytaczać kolejny raz tego samego kodu.

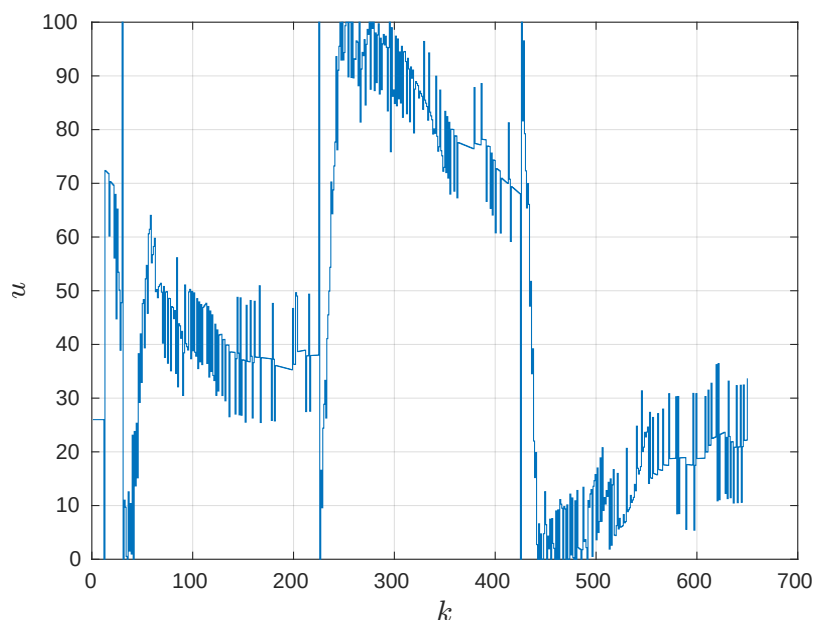
PID

Regulator PID został uruchomiony dla parametrów z 1. laboratorium tj.:

$$K = 18 \quad T_i = 40 \quad T_d = 10$$

Rys. 2.6: Wyjście y układu regulacji z konwencjonalnym regulatorem PID

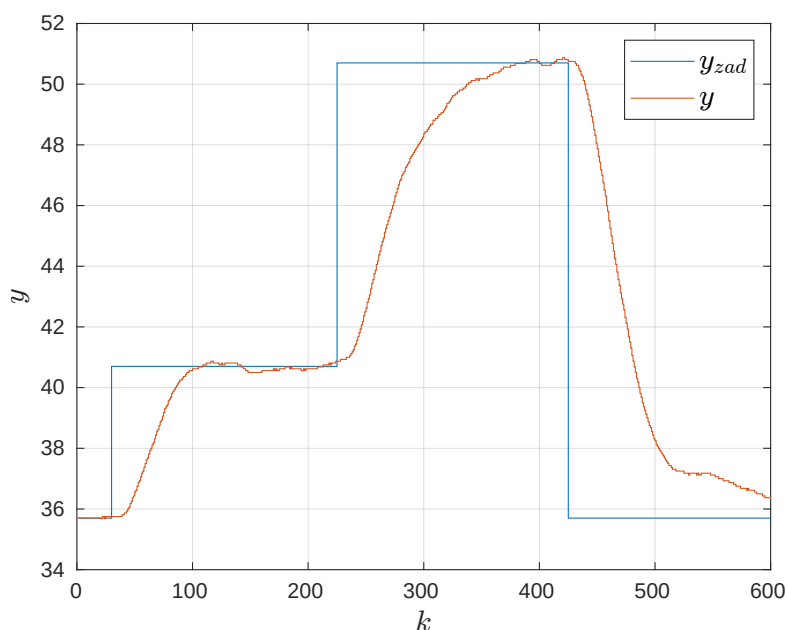
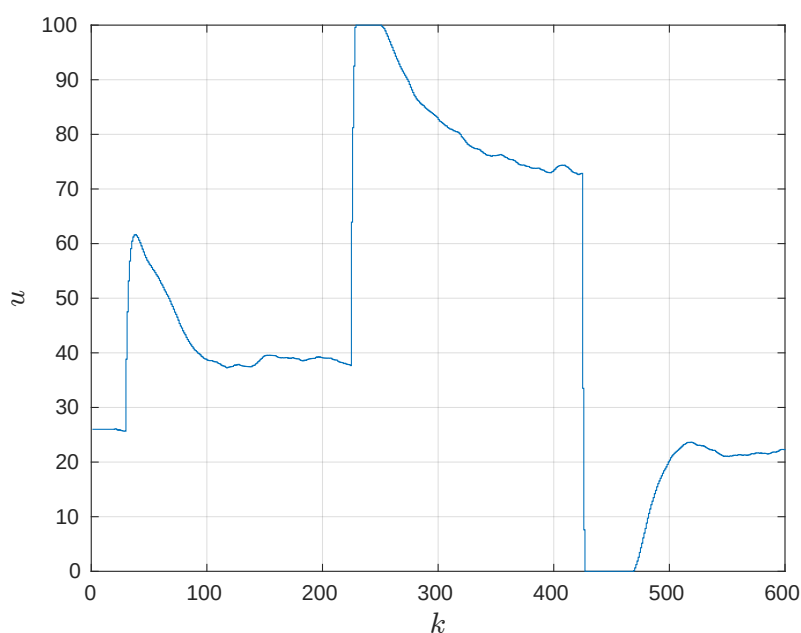
Jak widać na rysunku 2.6 konwencjonalny regulator PID działa akceptowalnie, ale zauważalny jest bardzo długi czas narastania sygnału wyjściowego y zanim osiągnie on wartość y_{zad} . To samo tyczy się zmniejszania się wartości wyjścia, gdy gwałtownie zmniejszymy wartość zadaną. Ocena jakościowa ma potwierdzenie w skumulowanym błędzie kwadratowym na przedziale całej trajektorii, który wyniósł $E = 16\,938,4083$.

Rys. 2.7: Sygnał sterujący u regulatora PID

DMC

Regulator DMC pracował z następującymi parametrami:

$$D = 600 \quad N = 50 \quad N_u = 6 \quad \lambda = 0,01$$

Rys. 2.8: Wyjście y układu regulacji z konwencjonalnym regulatorem DMCRys. 2.9: Sygnał sterujący u regulatora DMC

Na wykresie 2.8 widać, że konwencjonalny regulator DMC radził sobie lepiej z obiektem nieliniowym niż regulator PID. Sygnał wyjściowy był stabilniejszy i cechował się mniejszą ilością oscylacji, a także krótszym czasem narastania. Warte uwagi jest także sterowanie, które było spokojniejsze i tylko w jednym miejscu przekraczające górne ograniczenie. Spostrzeżenia te mają swoje pokrycie w wartości skumulowanego błędu kwadratowego, który tym razem wyniósł: $E = 12\,293,9360$.

2.2.2. Implementacja regulatorów rozmytych zgodnie z podejściem PDC

Podejście PDC (Parallel Distributed Compensation) polega na zastosowaniu kilku regulatorów lokalnych (czyli dostrojonych idealnie dla niewielkiego otoczenia konkretnego punktu pracy),

z których wartości sterowania są sumowane z odpowiednimi wagami zależącymi od wartości funkcji przynależności w danym punkcie charakterystyki. Zazwyczaj używa się tyłu regulatorów lokalnych, z ilu modeli lokalnych składa się model obiektu, czyli w tym przypadku z 3, co również wynika z treści polecenia.

Rozmyty PID

Zaimplementowany i przedstawiony na listingu 2.2 algorytm regulacji PID, oprócz tego, że jest to regulator rozmyty, różni się od implementacji z poprzednich laboratoriów także tym, że nie wykorzystano w nim wzorów wykorzystujących parametry r_0 , r_1 , r_2 , ale bardziej intuicyjne wzory wykorzystujące bezpośrednio parametry liniowego regulatora K , T_i oraz T_d :

$$\begin{aligned} u_p(k) &= Ke(k) \\ u_i(k) &= u_i(k-1) + \frac{KT}{T_i} \frac{e(k-1) + e(k)}{2} \\ u_d(k) &= KT_d \frac{e(k) - e(k-1)}{T} \\ u(k) &= u_p(k) + u_i(k) + u_d(k) \end{aligned} \quad (2.2)$$

gdzie T oznacza czas próbkowania w tym przypadku równy 1 s. Oczywiście wzory te dają identyczne wyniki jak te używające współczynników r_0 , r_1 , r_2 , jednak są bardziej intuicyjne, dlatego też zdecydowano się na ich zastosowanie w tym podejściu.

Listing 2.2: Implementacja rozmytego regulatora PID do sterowania stanowiskiem grzejąco-chłodzącym

```
clear all;

%% Configuration
% Initialize communication with the heating station
addpath('D:\SerialCommunication');
initSerialControl COM7

% General settings
start = 10;
kend = 650;

% PID parameters
working_points = [30 40 65];
K = [21.5 23 26];
Ti = [40 36 32];
Td = [10 11 11];

% Set trajectory
Ypp = 35;

step1_value = 40;
step2_value = 50;
step3_value = 60;

step1_time = 25;
step2_time = 220;
step3_time = 420;

yzad(1:step1_time) = Ypp;
yzad(step1_time:step2_time) = Ypp + 5;
yzad(step2_time:step3_time) = Ypp + 15;
yzad(step3_time:kend) = Ypp;

% Limits
u_min = 0;
```

```

u_max = 100;
e_sum = 0;

%% Files
% Preparing files
file_id = fopen(name, 'a');
fprintf(file_id, "u(k), yzad(k), y(k)\n");

buffer_size = 15;
buffer = zeros(buffer_size, 3);
buffer_index = 1;

% Make window with plots
figure;
hold on;
h_y = stairs(1:kstart, y(1:kstart), 'DisplayName', 'y');
h_y_zad = stairs(1:kstart, yzad(1:kstart), 'DisplayName', 'y\_zad');
hold off;

title("Wykres odpowiedzi stanowiska grzewczego");
xlabel("Iteracje [k]");
ylabel("Wartości");
legend;
grid on;

figure;
hold on;
h_u = stairs(1:kstart, u(1:kstart), 'DisplayName', 'u');
hold off;

title("Wykres sterowania stanowiska grzewczego");
xlabel("Iteracje [k]");
ylabel("Wartości");
legend;
grid on;

%% Algorithm
u = zeros(1, kend);
y = zeros(1, kend);
e = zeros(1, kend);
up = zeros(1, kend);
ui = zeros(1, kend);
ud = zeros(1, kend);

for k=start:kend
    y(k) = readMeasurements(1);

    % Compute error
    e(k) = yzad(k) - y(k);
    e_sum = e_sum + e(k)^2;

    membership_weights = membershipFunction(working_points, u(k-1));

    for i=1:num_regulators
        if membership_weights(i) == 0
            continue;
        end

        up(k) = K(i) * e(k);
        ui(k) = ui(k-1) + K(i) * (e(k-1) + e(k)) / (2 * Ti(i));
        ud(k) = K(i) * Td(i) * (e(k) - e(k-1));
        u_reg = up(k) + ui(k) + ud(k);

```

```

        u(k) = u(k) + membership_weights(i) * u_reg;
    end

    if(u(k) < u_min)
        u(k) = u_min;
    elseif(u(k) > u_max)
        u(k) = u_max;
    end

    % Control the physical object
    sendNonlinearControls(u(k))

    % Add data to buffer
    buffer(buffer_index, :) = [u(k), yzad(k), y(k)];
    buffer_index = buffer_index + 1;

    % Write data to file
    if buffer_index > buffer_size
        fprintf(file_id, '%f, %f, %f\n', buffer');
        buffer_index = 1;
    end

    % Update plots
    set(h_u, 'XData', 1:k, 'YData', u(1:k));
    set(h_y, 'XData', 1:k, 'YData', y(1:k));
    set(h_y_sim, 'XData', 1:k, 'YData', y_simulation(1:k));
    set(h_y_zad, 'XData', 1:k, 'YData', yzad(1:k));
    drawnow;

    waitForNewIteration();
end

% saving buffer if it isn't empty
if buffer_index > 1
    fprintf(file_id, '%f, %f\n', buffer(1:buffer_index-1, :));
end

fclose(file_id);
sendControls([1, 5], [50, 26]);

fprintf("Error: %f\r\n", e_sum);

```

Na początku skryptu wykonywane są typowe operacje inicjalizacyjne takie jak przygotowanie plików do zapisu danych oraz przygotowanie okien wykresów, a także zadanie trajektorii $y_{\text{zad}}(k)$. Następnie w pętli wykonywany jest algorytm rozmytego regulatora PID, który od konwencjonalnego algorytmu różni się tym, że na początku każdej iteracji są obliczane wartości funkcji przynależności na podstawie sygnału $u(k-1)$ za pomocą funkcji **membershipFunction** zaimplementowanej podczas projektu wykonanego przed blokiem 3. laboratoriów. Następnie w pętli są wyliczane wartości sterowań każdego z regulatorów lokalnych. Na ich podstawie jest wyznaczany wynikowy sygnał sterowania $u(k)$ będący sumą lokalnych sterowań przemnożonych przez odpowiednie wagi otrzymane z funkcji **membershipFunction**. Na koniec sterowanie wysyłane jest na obiekt laboratoryjny, wykresy i pliki są aktualizowane, po czym rozpoczyna się kolejna iteracja.

Rozmyty DMC

Na listingu 2.3 przedstawiono implementację rozmytego regulatora DMC do sterownia stanowiskiem grzejąco-chłodzącym:

Listing 2.3: Implementacja rozmytego regulatora DMC do sterowania stanowiskiem grzejąco-chłodzącym

```

%% Configuration
addpath('D:\SerialCommunication');
initSerialControl COM7
sendControls(1, 50);

start = 10;
kend = 800;

Upp = 26;
Ypp = 35.81;

y = zeros(kend, 1);
u = zeros(kend, 1);
e_sum = 0;

% Parameters to all DMC local regulators
D = 50;
N = 20;
Nu = 1;
lambda = 1;

% Set trajectory
step1_time = 25;
step2_time = 220;
step3_time = 420;

step1_value = 42;
step2_value = 52;

yzad(1:step1_time) = Ypp;
yzad(step1_time:step2_time) = step1_value;
yzad(step2_time:step3_time) = step2_value;
yzad(step3_time:kend) = Ypp;

%% Step responses
Upps = [26, 50, 65]; % u values from which steps were made during object
                        % identification
Us = [40, 55, 75]; % u values to which steps were made during object
                        % identification

num_regulators = length(Upps);
regulators = [];

for i=1:num_regulators
    regulators = [regulators localDMC_lab(N, Nu, D, lambda, Upps(i), ...
        Us(i), yzad, kend)];
end
%% Pliki
% File name to acquired data
test_number = 1;
name = "data/fuzzy_DMC_object_" + string(test_number) + ".csv";

% Preparing files
file_id = fopen(name, 'a');
fprintf(file_id, "u(k), y(k)");

buffer_size = 15;
buffer = zeros(buffer_size, 2);
buffer_index = 1;

% Make window with plots

```

```

figure;
hold on;
h_y = stairs(1:start, y_simulation(1:start), 'DisplayName', 'y');
h_y_zad = stairs(1:start, y_zad(1:start), 'DisplayName', 'y\_zad');
hold off;

title("Wykres sterowania i odpowiedzi stanowiska grzewczego");
xlabel("Iteracje [k]");
ylabel("Wartości");
legend;
grid on;

figure;
hold on;
h_u = stairs(1:start, u(1:start), 'DisplayName', 'u');
hold off;

title("Wykres sterowania i odpowiedzi stanowiska grzewczego");
xlabel("Iteracje [k]");
ylabel("Wartości");
legend;
grid on;

% Main loop
for k=start:kend
    y(k) = readMeasurements(1);

    % Compute error
    ek = yzad(k) - y(k);
    e_sum = e_sum + ek^2;

    membership_weights = membershipFunction(working_points, u(k-1));

    % Get u from local regulators
    for i=1:num_regulators
        if membership_weights(i) == 0
            continue;
        end

        u_reg = regulators(i).iterationDMC(k, y(k));
        u(k) = u(k) + membership_weights(i) * u_reg;
    end

    % Update local regulators
    for i=1:num_regulators
        regulators(i) = regulators(i).updateAfterIteration(u(k), k);
    end

    % Control the physical object
    sendNonlinearControls(u(k))

    % Add data to buffer
    buffer(buffer_index, :) = [u(k), y(k)];
    buffer_index = buffer_index + 1;

    % Write data to file
    if buffer_index > buffer_size
        fprintf(file_id, '%f, %f\n', buffer');
        buffer_index = 1;
    end

    % Update plots
    set(h_u, 'XData', 1:k, 'YData', u(1:k));

```



```

    set(h_y, 'XData', 1:k, 'YData', y(1:k));
    drawnow;

    waitForNewIteration();
end

% saving buffer if it isn't empty
if buffer_index > 1
    fprintf(file_id, '%f, %f\n', buffer(1:buffer_index-1, :));
end

fclose(file_id);
sendControlsToG1AndDisturbance(Upp, 0);

fprintf("Error: %f\r\n", e_sum);

```

Jest to skrypt podobny do tych implementowanych w ramach poprzednich bloków laboratoryjnych. Pierwszą z istotnych różnic jest podanie dwóch wektorów **Upps** z wartościami, z których wykonywane były skoki wartości u przeprowadzane podczas identyfikacji obiektu, oraz **Us** z wartościami u , do których były one wykonywane. Wartości te są następnie wykorzystywane do ustawienia odpowiednich warunków początkowych solvera używanego do wykonania aproksymacji.

Kolejną z różnic jest tworzenie w pętli kolejnych instancji klasy **localDMC_lab** (listing 2.4), która jest podobna do klasy **localDMC** zaimplementowanej w ramach projektu przedlaboratoryjnego i która realizuje pojedynczy lokalny regulator DMC.

Właściwy proces regulacji zaczyna się w sekcji oznaczonej komentarzem **% Main loop**. W każdej iteracji pętli są wyliczane wartości funkcji przynależności każdego z regulatorów za pomocą funkcji **membershipFunction** zaimplementowanej w ramach projektu (listing 1.6). Następnie w pętli są wyliczane wartości sterowań przez każdy z regulatorów lokalnych, po czym wartości z ich wyjść są sumowane z odpowiednimi wagami wynikającymi z działania funkcji **membershipFunction** w wynikowe sterowanie $u(k)$, które zadawane jest na obiekt z użyciem funkcji **sendNonlinearControls**. Na końcu każdej iteracji są aktualizowane wykresy oraz dodawane są dane do bufora służącego do zapisu danych do pliku.

Implementacja wspomnianej wcześniej klasy **localDMC_lab** została przedstawiona na listingu 2.4:

Listing 2.4: Implementacja klasy **localDMC_lab**

```

classdef localDMC_lab
    properties
        % Constraints
        u_min = -1
        u_max = 1
        % Trajectory
        yzad
        % Regulator parameters
        deltauk_p
        u
        Ke
        Ku
        % Horizont
        D
        Upp
        U
    end

    methods
        function obj = localDMC_lab(N, Nu, D, lambda, Upp, U, yzad, kend)
            obj.D = D;

```

```

obj.yzad = yzad;
obj.Upp = Upp;
obj.U = U;

% Step response
[s, Ypp] = step_response_lab(D, Upp, U);

% Fill M matrix
M = zeros(N, Nu);
for i=1:Nu
    M(i:end,i)=s(1:N-i + 1);
end

% Fill MP matrix
MP = zeros(N, D-1);
for i = 1:N
    for j = 1:D-1
        if i+j <= D
            MP(i, j) = s(i+j) - s(j);
        else
            MP(i, j) = s(D) - s(j);
        end
    end
end

% Regulator parameters
I = eye(Nu);
K = ((M'*M+lambda*I)^(-1))*M';
obj.Ku = K(1,:)*MP;
obj.Ke = sum(K(1, :));

% Variables initialization
obj.u = zeros(kend, 1);
obj.deltauk_p = zeros(D-1, 1);
end

function u_current = iterationDMC(obj, k, y_current)
    % Compute error
    ek = obj.yzad(k) - y_current;

    % Compute deltau variable for given control horizon
    deltau = obj.Ke * ek - obj.Ku * obj.deltauk_p;

    % Manipulate variable for time k
    u_current = obj.u(k-1) + deltau;

    if(u_current < obj.u_min)
        u_current = obj.u_min;
    elseif(u_current > obj.u_max)
        u_current = obj.u_max;
    end
end

function obj = updateAfterIteration(obj, last_u, k)
    % Back deltau window
    for n=obj.D-1:-1:2
        obj.deltauk_p(n) = obj.deltauk_p(n-1);
    end

    obj.u(k) = last_u;
    obj.deltauk_p(1) = obj.u(k) - obj.u(k-1);
end
end

```

```
end
```

Jak można zauważyć, różnica między analogiczną klasą wykorzystywaną w ramach projektu polega na innym sposobie otrzymywania wektorów odpowiedzi skokowych s . W tym przypadku używa się funkcji `step_response_lab` przedstawionej na listingu 2.5. Pobiera ona z odpowiedniego pliku CSV dane zebrane ze stanowiska grzejąco-chłodzącego podczas wykonywania odpowiednich skoków wartości $u(k)$ z `Upp` do `U`, które to są podawane jako argumenty funkcji. Następnie funkcja ta wykonuje aproksymację używając solvera `approximation`, który był zaimplementowany podczas pierwszego bloku laboratoryjnego i którego opis tym razem nie zostanie już przytoczony. Na koniec obliczana jest wartość wektora współczynników odpowiedzi skokowej s na horyzoncie dynamiki D przez funkcję `step_response`, która także została zaimplementowana na poprzednich blokach laboratoryjnych i która podobnie nie zostanie ponownie opisana, aby uniknąć powielania tych samych treści w kolejnych sprawozdaniach.

Listing 2.5: Implementacja funkcji `step_response_lab`

```
function [s, y_out] = step_response_lab(D, Upp, U)
    name1 = "data/dmc_u=" + string(Upp) + ".csv";
    raw_data1 = load(name1);

    [xopt1, td1] = approximation(U, Upp, raw_data1);
    K1 = xopt1(1);
    T11 = xopt1(2);
    T21 = xopt1(3);

    s = step_response(0, 0, D+1, K1, T11, T21, td1);
    s = s(2:end);
    y_out = raw_data1(end,1);
end
```

2.2.3. Testowanie regulatorów rozmytych

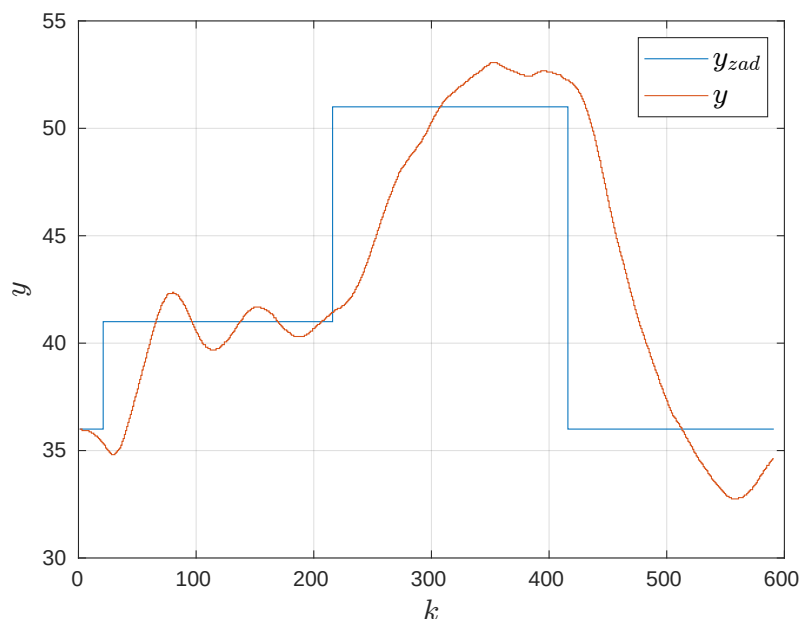
Rozmyty PID

Strojenie rozmytego regulatora PID jest zadaniem trudnym i wymagającym czasu i doświadczenia. W ogólnym przypadku może ono polegać na dostrojeniu każdego z regulatorów lokalnych z osobna dla niedużych skoków $y_{\text{zad}}(k)$ w okolicy ich punktów pracy. Po tych operacjach rozmyty regulator PID powinien działać dobrze. Do strojenia lokalnych regulatorów można użyć typowych znanych metod jak na przykład metoda Zieglera-Nicholsa. Jednakże wszystkie opracowane metody strojenia regulatorów PID wymagają jego dodatkowego dostrojenia we własnym zakresie, aby osiągnąć dobrą jakość regulacji. W tym przypadku posiadano już całkiem dobrze działające nastawy z 1. bloku laboratoryjnego, czyli $K = 18$, $T_i = 40$ oraz $T_d = 10$, co może być punktem wyjścia dla nastaw wszystkich regulatorów lokalnych ze względu na fakt, że nieliniowość obiektu nie jest duża, a więc oczywiste jest to, że parametry kolejnych regulatorów lokalnych nie będą się od siebie mocno różnić.

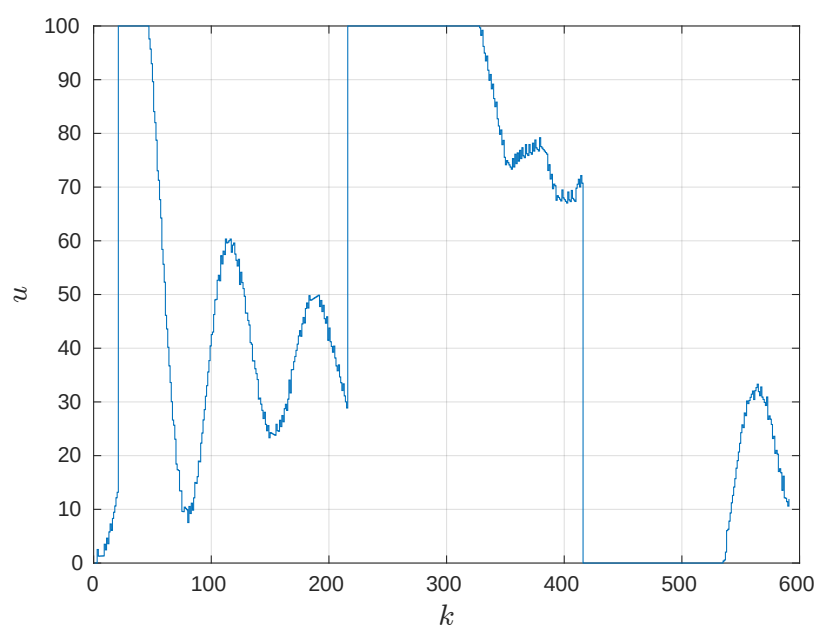
Na rysunku 2.6 widać, że konwencjonalny regulator PID miał dość długie czasy narastania sygnału $y(k)$. Aby przyspieszyć działanie regulatora w tym przypadku zdecydowano się na nieznaczną zmianę parametrów, z których rozpoczęto proces strojenia kolejnych regulatorów: $K = 21,5$, $T_i = 40$ i $T_d = 10$, takie też parametry przyjęto dla pierwszego regulatora lokalnego.

Nieliniowość charakterystyki nie jest duża, jednakże jej wzmocnienie spada wraz z wartością u , dlatego też rozpoczęto eksperymenty, w których zwiększano nieznacznie wartość wzmocnienia K dla kolejnych regulatorów lokalnych, a następnie testowano regulator rozmyty na rzeczywistym obiekcie. W ten sposób uzyskano przyzwoitą pracę regulatora dla kolejnych wartości wzmocnienia: $K_1 = 21,5$, $K_2 = 23$, $K_3 = 26$, gdzie indeks dolny oznacza numer regulatora lokalnego. Następnym krokiem była zmiana okresów całkowania i różniczkowania. Dla najlepszych nastaw

regulatora parametry te wynosiły odpowiednio $T_{i1} = 40$, $T_{i2} = 36$, $T_{i3} = 32$ oraz $T_{d1} = 10$, $T_{d2} = 11$, $T_{d3} = 11$. Dla tak przyjętych nastaw regulator działał w sposób przedstawiony na wykresach $y(k)$ i $u(k)$ (wykresy 2.10 oraz 2.11).



Rys. 2.10: Wyjście y układu regulacji z rozmytym regulatorem PID



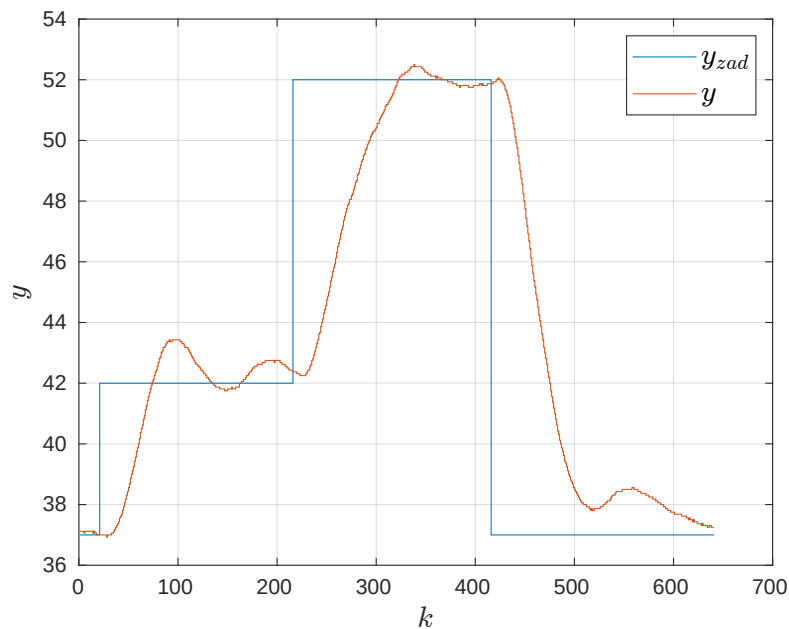
Rys. 2.11: Sygnał sterujący u rozmytego regulatora PID

Porównując błąd kwadratowy rozmytego regulatora PID wynoszący $E = 13\,724,3267$ z błędem konwencjonalnego regulatora PID ($E = 16\,938,4083$) można stwierdzić, że regulator działa lepiej. Niewielki ujemny skok wartości y na początku wykresu wynika z trochę zbyt wczesnego włączenia regulatora, gdy stan początkowy nie był jeszcze idealnie ustalony. Następnie widać, że wartość y szybciej zbiega do wartości zadanej niż miało to miejsce w konwencjonalnym regulatorze PID. Niestety jest to okupione przeregulowaniem, które jest jednak akceptowalne, mając na uwadze chęć szybkiego osiągnięcia wartości zadanej. W tym miejscu oraz w przypadku drugie-

go skoku $y_{\text{zad}}(k)$ widać niekorzystne zjawisko windup, które polega na tym, że uchyb podlega całkowaniu, a w przypadku, gdy wyjście regulatora u się nasyci, algorytm wylicza wartość u większą niż może zapewnić urządzenie, przez co wartość całki nadal rośnie, co sprawia, że w tym przypadku przeregulowanie jest znacznie dłuższe niż by było, gdyby wyjście się nie nasyciło. Należałoby dlatego zaimplementować mechanizm anti-windup, jednakże na ograniczony czas trwania laboratorium nie zdążono tego zrobić. Niestety zjawisko te utrudnia realne ocenienie jakości regulacji rozmytego regulatora PID. Jednakże na pierwszy rzut oka można stwierdzić, że jest on szybszy, zwłaszcza dla większych wartości zadanych, co jest oczywiste, skoro wzmocnienie obiektu dla dużych u jest mniejsze, lecz kolejne coraz to większe wzmocnienia lokalnych regulatorów kompensują ten problem.

Jednakże eksperymenty te pokazały, że strojenie rozmytego regulatora PID nie jest zadaniem łatwym i szybkim. Brak dostatecznego czasu w tym przypadku uniemożliwił uzyskanie regulatora o parametrach, które byłyby bardzo znacząco lepsze od regulatora konwencjonalnego. Mimo to można stwierdzić, że regulatory rozmyte są bardzo dobre i użyteczne w przypadku procesów nieliniowych. Warto jednak mieć na uwadze to, że trzeba zawsze realnie i optymalnie dobierać narzędzia do określonych zadań. Nie zawsze nieliniowość obiektu oznacza, że niezbędny jest regulator rozmyty. Czasami, gdy nieliniowość obiektu nie jest duża, jak to było na przykład w tym przypadku, lepsze może się okazać wykorzystanie regulatora konwencjonalnego, gdyż jego implementacja jest prostsza, wymagająca mniej obliczeń oraz jest łatwiejszy w regulacji, przez co można łatwo znaleźć zadowalające parametry. Regulatory rozmyte przez trudności z ich skrojeniem czasem mogą przynieść nawet gorszy rezultat, gdyż uniemożliwia to znalezienie ich idealnych nastaw.

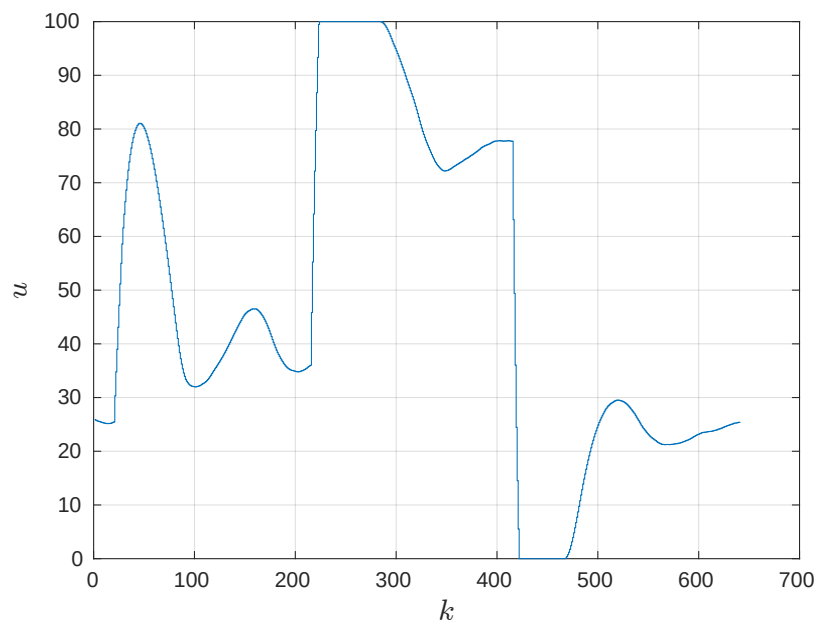
Rozmyty DMC



Rys. 2.12: Wyjście y układu regulacji z rozmytym regulatorem DMC o parametrach $D = N = N_u = 600$, $\lambda = 1$

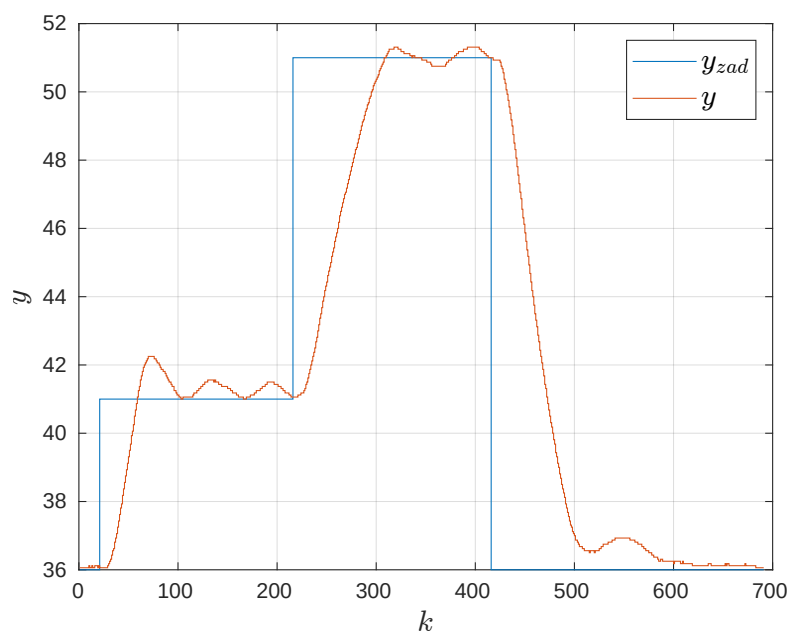
Na wykresach 2.12 oraz 2.13 przedstawiono działanie regulatora DMC w wersji rozmytej, gdzie według polecenia w zadaniu wszystkie regulatory lokalne mają te same parametry, w których wszystkie horyzonty są takie same $D = N = N_u = 600$ (czyli takie jak był horyzont

dynamiki w testowanym regulatorze konwencjonalnym), a kara za zbyt szybkie przyrosty sygnału sterowania $\lambda = 1$.



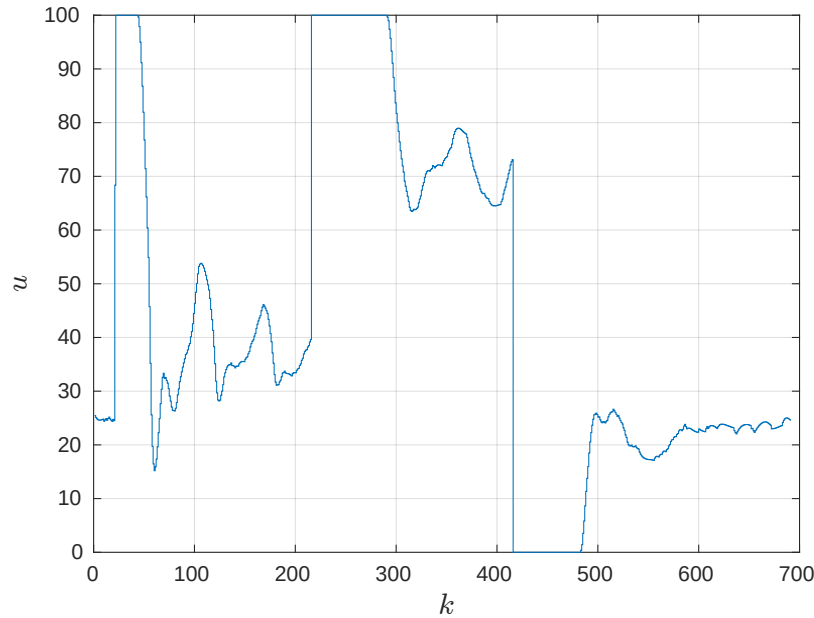
Rys. 2.13: Sygnał sterujący u rozmytego regulatora DMC o parametrach $D = N = N_u = 600$, $\lambda = 1$

Wartość błędu kwadratowego w tym przypadku wynosi $E = 13\,203,4983$, co już jest wynikiem nieznacznie lepszym od rozmytego regulatora PID, w którym błąd wynosił $E = 13\,724,3267$, mimo że regulator DMC nie miał w tym przypadku jeszcze dobranych odpowiednio parametrów. Na pierwszy rzut oka widać, że przeregulowania są mniejsze i trwają krócej, zwłaszcza w przypadku drugiego skoku wartości zadanej, po którym w przypadku regulatora PID widoczne było wystąpienie zjawiska windup'u.



Rys. 2.14: Wyjście y układu regulacji z rozmytym regulatorem DMC o parametrach $D = N = N_u = 600$, $\lambda = 0,01$

Następnym krokiem było dobranie wartości parametru λ przy zachowanych takich samych wszystkich horyzontach dynamiki, predykcji i sterowania jak poprzednio, zgodnie z treścią polecenia. Dobrano najlepszą wartość parametru λ jako 0,01. Jest to wartość przy której regulator działa możliwie szybko, ale jeszcze nie wpada w oscylacje. Wykresy wartości wyjściowej procesu oraz sterowania dla regulatora rozmytego DMC o tych parametrach zostały zaprezentowane na wykresach 2.14 oraz 2.15.



Rys. 2.15: Sygnał sterujący u rozmytego regulatora DMC o parametrach $D = N = N_u = 600$, $\lambda = 0,01$

W tym przypadku błąd kwadratowy zmalał z $E = 13\,203,4983$ dla regulatora rozmytego o $\lambda = 1$ do wartości $E = 11\,709,6638$. Widoczne jest, że w tym przypadku wartość y rośnie możliwie szybko - szybszy wzrost nie jest możliwy, gdyż jak widać, sygnał sterujący osiąga już w tych miejscach maksymalne wartości. Przeregulowania są także mniejsze niż poprzednio. W ten sposób osiągnięto sporą poprawę względem regulatora konwencjonalnego, w którym co prawda nie występowały przeregulowania, ale który działał zdecydowanie wolniej. Natomiast w tym przypadku przeregulowania mają niewielkie wartości i są w pełni tolerowane. Poprawa szybkości działania regulatora DMC wynika z tego, że regulator rozmyty uwzględnia w modelu zmiany wzmocnienia obiektu dla większych wartości u , na co może z wyprzedzeniem reagować, wystawiając jeszcze większą wartość sterownia.

Z przeprowadzonych eksperymentów wynika, że stosowanie rozmytych regulatorów zazwyczaj ma sens w przypadku procesów nieliniowych, lecz nie zawsze, wszystko zależy od stopnia nieliniowości. W przypadku tego obiektu udało się uzyskać regulatory o lepszych parametrach, jednak ze względu na dość niewielką nieliniowość obiektu, efekty nie były spektakularne.